

Geospatial Data Analysis with Python

Python programming for geospatial data handling, processing, visualization &
workflow automation

An open educational resource

Dr. Suborna Ahmed · UBC Faculty of Forestry & Environmental Stewardship

Contents

About this book

Chapter 1 — Python Foundations

Chapter 2 — Programming Logic & Reusable Code

Chapter 3 — Tabular Data with Pandas

Chapter 4 — Vector Data with GeoPandas & Shapely

Chapter 5 — CRS, Files & Data Quality

Chapter 6 — Consolidation

Chapter 7 — ArcPy: Tools & Environments

Chapter 8 — ArcPy: Automation

Chapter 9 — Raster Data with NumPy & Rasterio

Chapter 10 — Visualization & Communication

Chapter 11 — Introductory Machine Learning

About this book

This open educational resource teaches **Python programming for geospatial data analysis**, starting from no assumed programming experience. You'll learn Python fundamentals with small geospatial examples, then work with tabular, vector, and raster data; automate GIS tasks; produce clear maps; and finish with an introduction to machine learning on satellite imagery.

Two environmental datasets recur throughout: the **Malcolm Knapp Research Forest** in British Columbia and the **mangrove coast of northwest Madagascar**, alongside Landsat imagery.

How to use this book

- Each chapter is a hands-on lesson: read the explanation, run the code, and try the **Quick check** and **Try it yourself** prompts (answers are collapsible).
- Most chapters run in an **open-source Python environment** (Pandas, GeoPandas, Shapely, Rasterio, NumPy, Matplotlib, scikit-learn). Two chapters use **ArcPy**, which requires ArcGIS Pro.
- Each chapter ends with **practice exercises** to work through on your own.

About the author

Dr. Suborna Ahmed — Associate Professor of Teaching, Department of Forest Resources Management, Faculty of Forestry & Environmental Stewardship, University of British Columbia. She teaches data analytics, computing, forest biometrics, and quantitative methods for forestry and natural-resources students, and develops open educational resources using real forestry datasets.

Companion book

See the companion resource, [*Modern Data Analytics for Forestry and Natural Resources*](#) (efficient coding, data analysis, and AI-assisted debugging, in R & Quarto).

License

Content is licensed under [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#).
Code is under the MIT licence.

How to cite

Ahmed, S. (2026). *Geospatial Data Analysis with Python*. UBC Faculty of Forestry & Environmental Stewardship. CC BY 4.0.

Chapter 1 — Python Foundations

This is your **first notebook**. We learn the building blocks of Python using tiny, real geospatial examples — coordinates and shapes for two places we will return to throughout: the **Malcolm Knapp Research Forest (MKRF)** in British Columbia and the **mangrove coast of NW Madagascar**.

No prior Python experience is assumed. Run each code cell, read the output, and try the *Try it yourself* prompts.

Learning outcomes

By the end of this chapter you should be able to:

1. Run and edit **code** and **Markdown** cells in Jupyter, and write comments.
2. Create and inspect **variables** (numbers, strings, booleans).
3. Build and index **lists, tuples, and dictionaries**.
4. Construct a **file path** with `pathlib`.
5. **Import** a package and use it.
6. Make a simple **Shapely** `Point`, `LineString`, and `Polygon` from coordinates.

How this notebook works

- **Code cells** (grey) contain Python. Click one and press **Shift + Enter** to run it.
- **Markdown cells** (like this one) are text/instructions.
- Lines starting with `#` inside code are **comments** — notes for humans, ignored by Python.
- Make sure the kernel (top-right) says **Python (gem530)**. If not, see the environment setup in `environments/README.md`.

0. First run — check your setup

Run the cell below. If you see a Python version printed, you are ready.

```
import sys
print("Python version:", sys.version.split()[0])
print("If you can read this, your notebook is working.")
```

```
Python version: 3.11.15
If you can read this, your notebook is working.
```

1. Variables: describing a field site

A **variable** is a name that stores a value. Let's describe the MKRF research forest. Notice the different **types**: text (`str`), whole numbers (`int`), decimals (`float`), and true/false (`bool`).

```
# A variable = name on the left, value on the right
site_name  = "Malcolm Knapp Research Forest"  # str (text, in quotes)
latitude   = 49.27                            # float (decimal degrees
North)
longitude  = -122.57                          # float (decimal degrees; West
is negative)
elevation_m = 150                             # int (whole number, metres)
is_protected = True                           # bool (True or False)

print(site_name + ":", latitude, "latitude,", longitude, "longitude")
```

```
Malcolm Knapp Research Forest: 49.27 latitude, -122.57 longitude
```

Use `type()` to ask Python what type a value is:

```
print(type(site_name))
print(type(latitude))
print(type(elevation_m))
print(type(is_protected))
```

```
<class 'str'>
<class 'float'>
```

```
<class 'int'>
<class 'bool'>
```

Try it yourself. In a new cell, create variables for a second site — the Madagascar mangrove coast is near `-13.35, 48.55`. Give it a `site_name`, `latitude`, `longitude`, and a boolean like `has_mangroves = True`, then print them.

▼ **Show answer**

```
site_name = "Madagascar mangrove coast"
latitude = -13.35
longitude = 48.55
has_mangroves = True
print(site_name, latitude, longitude, has_mangroves)
```

 Quick check — types

`elevation_m = 150` and `latitude = 49.27`. Which is an `int`, and which is a `float`?

▼ ► **Show answer**

`elevation_m` is an **int** (whole number); `latitude` is a **float** (it has a decimal). Confirm with `type(elevation_m)`.

🌲 **Why code this?** Every field measurement — a tree's height, a plot's coordinates, whether a stand is protected — is a *variable*. Naming values clearly is the first step to a **reproducible** forest analysis instead of a maze of unlabelled spreadsheet cells.

2. Operators: doing things with values

Arithmetic (`+` `-` `*` `/`), **comparison** (`>` `<` `==`), and **logical** (`and` `or` `not`) operators let us compute and ask questions about our data.

```
# Arithmetic
elevation_ft = elevation_m * 3.281
print("Elevation in feet:", round(elevation_ft, 1))

# Comparison -> gives a boolean
in_northern_hemisphere = latitude > 0
print("Northern hemisphere?", in_northern_hemisphere)

# Logical: combine conditions
print("Protected AND northern?", is_protected and in_northern_hemisphere)
```

```
Elevation in feet: 492.2
Northern hemisphere? True
Protected AND northern? True
```

Try it yourself. Is the Madagascar site in the northern hemisphere? Write a comparison using its latitude (it is negative — south of the equator).

▼ Show answer

```
madagascar_lat = -13.35
print(madagascar_lat > 0)    # False — it is south of the equator
```

 **Forestry example.** Comparison and logic let you *flag data automatically*: `dbh_cm > 50` finds the big trees; `slope > 30` and `forested` finds erosion-risk stands. You'll run these checks over **thousands of records** instead of scanning rows by eye.

3. Data structures: grouping many values

Real geospatial work involves *collections* of values. Three you will use constantly:

Structure	Looks like	Use it for	Changeable?
list	<code>[a, b, c]</code>	an ordered, editable collection	yes

Structure	Looks like	Use it for	Changeable?
tuple	(x, y)	a fixed pair/record (e.g. a coordinate)	no
dict	{"key": value}	labelled attributes of one thing	yes

```
# A tuple: one coordinate pair (longitude, latitude) -- fixed
mkrf_point = (-122.57, 49.27)

# A list: several plot coordinates in MKRF -- ordered and editable
plot_coords = [
    (-122.573, 49.271),
    (-122.569, 49.268),
    (-122.566, 49.274),
]

# A dictionary: labelled attributes of one site
site = {
    "name": "MKRF",
    "lon": -122.57,
    "lat": 49.27,
    "elevation_m": 150,
    "protected": True,
}

print("Number of plots:", len(plot_coords))
print("Site name from dict:", site["name"])
```

```
Number of plots: 3
Site name from dict: MKRF
```

Indexing pulls items out. Python counts from 0:

```
print("First plot:", plot_coords[0])    # index 0 = first
print("Last plot: ", plot_coords[-1])   # -1 = last
print("Longitude of first plot:", plot_coords[0][0]) # first item, then its
first value

# Lists are editable: add a new plot
```

```
plot_coords.append((-122.571, 49.269))
print("Now there are", len(plot_coords), "plots")
```

```
First plot: (-122.573, 49.271)
Last plot:  (-122.566, 49.274)
Longitude of first plot: -122.573
Now there are 4 plots
```

Try it yourself. Add a "country" key to the `site` dictionary with the value "Canada", then print the whole dictionary.

▼ **Show answer**

```
site["country"] = "Canada"
print(site)
```

 Quick check — indexing

For `plots = [(-122.57, 49.27), (-122.55, 49.29)]`, what does `plots[1][0]` give?

▼ **Show answer**

-122.55 — the **second** tuple (index 1), then its **first** value (index 0), the longitude. Remember indexing starts at 0.

 **Why code this?** A forest inventory is a list of plot dictionaries; a species-code table is a dictionary. Lists and dicts hold **many records at once** — no separate variable per tree — which is exactly the idea behind Pandas (Chapter 3).

4. File paths with `pathlib`

Your data lives in files. `pathlib` builds paths that work on Windows, Mac, and Linux. Use the `/` operator to join folders — cleaner and safer than typing `"data\\sites.csv"` by hand.

```
from pathlib import Path

data_dir = Path("data")          # a folder (relative path)
sites_file = data_dir / "sites.csv" # join folder + filename

print("Full path:", sites_file)
print("File name:", sites_file.name)
print("Extension:", sites_file.suffix)
```

```
Full path: data\sites.csv
File name: sites.csv
Extension: .csv
```

We will treat *relative paths* (like `data/sites.csv`) as the reproducible default and revisit **why** in **Chapter 5 (files & data quality)**.

🌲 **Why code this?** Environmental data lives in *hundreds* of files (shapefiles, rasters, CSVs). Building paths in code means the same analysis re-runs on a new folder of survey data, on any computer — no manual re-typing, no broken links.

5. Importing packages

Python's power comes from **packages**. You `import` a package, then use its tools with a dot: `package.tool()`. Here is the built-in `math` package.

```
import math

# straight-line distance between two of our plots (in DEGREES for now)
d = math.dist(plot_coords[0], plot_coords[1])
print("Planar distance:", round(d, 5), "degrees")
```

```
Planar distance: 0.005 degrees
```

Notice the distance is in **degrees**, which is not a real-world length. Turning coordinates into metres needs a **coordinate reference system** — the topic of **Chapter 5**. For now, just notice the problem exists.

🌲 **Why code this?** Nobody re-writes geospatial math from scratch. Importing packages (NumPy, GeoPandas, Rasterio) lets you build on tested tools — the difference between an afternoon and a month on a mangrove-loss analysis.

6. Your first geometries: Shapely

A coordinate pair is just numbers. **Shapely** turns coordinates into *geometry objects* — `Point`, `LineString`, `Polygon` — the same objects that **GeoPandas** (Chapter 4) uses for whole datasets. This is a *tiny* first taste.

```
from shapely.geometry import Point, LineString, Polygon

# A Point: one location (lon, lat)
mkrf = Point(-122.57, 49.27)

# A LineString: an ordered path (e.g. a short trail) through several points
trail = LineString([(-122.573, 49.271), (-122.569, 49.268), (-122.566,
49.274)])

# A Polygon: a closed area (first and last point join automatically)
stand = Polygon([(-122.575, 49.270), (-122.565, 49.270),
                 (-122.565, 49.276), (-122.575, 49.276)])

print("Point:      ", mkrf)
print("Trail length:", round(trail.length, 5), "degrees")
print("Stand area: ", round(stand.area, 6), "square degrees")
```

```
Point:      POINT (-122.57 49.27)
Trail length: 0.01171 degrees
Stand area:  6e-05 square degrees
```

In Jupyter, a Shapely geometry draws itself. Run the next cell to see the polygon (its shape appears as a small picture):

```
stand # displaying a geometry shows its shape
```



Try it yourself. Make a **triangle** `Polygon` from any three coordinates near the Madagascar site (around `48.55, -13.35`), and display it. Remember Shapely expects `(lon, lat)` order.

▼ ✓ **Show answer**

```
from shapely.geometry import Polygon
triangle = Polygon([(48.55, -13.35), (48.60, -13.35), (48.57, -13.30)]) #
(lon, lat)!
print(triangle)
triangle
```

🌲 **Forestry example.** A sample plot is a `Point`, a stream or trail is a `LineString`, and a harvest block or mangrove patch is a `Polygon`. Turning coordinates into geometry objects is what later lets you **measure areas, find what's inside a reserve, and map change over time.**

✓ **Quick check — geometry order**

To place a point at **latitude 49.3, longitude -122.6**, do you write `Point(49.3, -122.6)` or `Point(-122.6, 49.3)`?

▼ ► **Show answer**

`Point(-122.6, 49.3)` — Shapely takes **(lon, lat)**. Writing `(lat, lon)` puts the point in the wrong place (a classic bug).

Hands-on activity — Build Your Field Site (~20 min)

Turn a **real place you care about** into Python data and your first geometry. Work individually; edit the starter cell below.

Task 1. Create variables for your site: `site_name`, `lat`, `lon`, `protected`. **Task 2.** Make a list `neighbours` of 2–3 nearby `(lon, lat)` coordinates, and a dictionary `site` with keys `name`, `lat`, `lon`. **Task 3.** Make a Shapely `Point` for your site and a `LineString` through your `neighbours`; print the line's `.length` and display the point.

```
# ---- Your turn: edit the values, then run ----
site_name = "..."          # e.g. "Malcolm Knapp Research Forest"
lat, lon = 0.0, 0.0         # your coordinates (decimal degrees)
protected = False

# Task 2: a list of neighbours and a dict
neighbours = [(lon, lat), (lon, lat)]      # <- change these!
site = {"name": site_name, "lat": lat, "lon": lon}

# Task 3: make your geometries (fill these in)
# from shapely.geometry import Point, LineString
# here = Point( ... )
# path = LineString( ... )

print(site)
print(neighbours)
```

```
{'name': '...', 'lat': 0.0, 'lon': 0.0}
[(0.0, 0.0), (0.0, 0.0)]
```

Activity solutions

Have a go first, then open each one as we talk it through.

▼ ► Task 1 — variables

```
site_name = "Malcolm Knapp Research Forest"
lat, lon = 49.27, -122.57
protected = True
print(site_name, lat, lon, protected)
```

▼ ► Task 2 — list & dict

```
neighbours = [(-122.57, 49.27), (-122.55, 49.29), (-122.52, 49.31)]
site = {"name": site_name, "lat": lat, "lon": lon}
print(neighbours)
print(site)
```

▼ ► Task 3 — geometries

```
from shapely.geometry import Point, LineString
here = Point(lon, lat) # (lon, lat)!
path = LineString(neighbours)
print("Point:", here)
print("Line length (degrees):", round(path.length, 4))
here
```

Packages used this chapter

(what it does · why we use it · when · how it connects to geospatial work)

- **Jupyter Notebook** — interactive code + notes + output in one document. *Why:* run geospatial code step by step and keep a readable record. *When:* every chapter. *Geospatial link:* the workspace where all other tools run; maps render inline.
- **Python standard library** (built in) — core language + helpers like `math`. *Why:* the foundation everything builds on. *When:* always.
- **pathlib** (standard library) — build file paths safely. *Why:* geospatial work is file-heavy; reproducible paths prevent "file not found" errors. *When:* any time you read/write data. *Geospatial link:* every shapefile/raster/CSV is opened by a path.

- **Shapely** — geometry objects (Point, LineString, Polygon). *Why*: represent real features as objects you can measure. *When*: introduced here; used fully via GeoPandas in Chapter 4. *Geospatial link*: the geometry engine under GeoDataFrames.

See the full [Package Guide](#) for more.

Why this matters

Today's pieces look small, but they are the **vocabulary of every geospatial workflow**: a forest inventory is a list of plot dictionaries, a satellite image is arrays of numbers, and a conservation map is Shapely geometries with attributes. Master these and the rest of the book is *combining* them.

Career connection. GIS analysts, remote-sensing scientists, and conservation data specialists write exactly this code every day — reading coordinates, organising attributes, and turning them into maps and decisions.

Concept check

1. What will `type(49.27)` say — and why is it different from `type(49)` ?
2. `coords = [(-122.6, 49.3), (-122.5, 49.4)]`. How do you get the **latitude** of the second point? (Careful with the order!)

Vote/show hands first, then reveal:

▼ ► Answer 1

`<class 'float'>`, because `49.27` has a decimal point. `type(49)` is `<class 'int'>` (a whole number).

▼ ► Answer 2

`coords[1][1]` — the second tuple (index 1), then its second value (the latitude).
`coords[1][0]` would give the longitude.

Common errors this chapter (and the fix)

Message you see	Likely cause	Fix
<code>NameError: name 'x' is not defined</code>	used a variable before creating it, or a typo	define it first; check spelling
<code>IndexError: list index out of range</code>	asked for <code>list[3]</code> when it holds fewer items	indexing starts at 0 ; check <code>len()</code>
<code>SyntaxError</code>	missing quote, bracket, or <code>:</code>	read the arrow in the message; check the line above
point in the wrong place	swapped lon/lat	Shapely wants <code>(lon, lat)</code>

Quick reference — Chapter 1 cheat sheet

```

name = "MKRF"           # string (str)
lat  = 49.27            # decimal (float)
coords = [(-122.6, 49.3)] # list of (lon, lat) tuples
site = {"name": "MKRF"}  # dictionary
coords[0]                # first item (index starts at 0)
site["name"]              # dict value by key
from pathlib import Path
Path("data") / "sites.csv" # build a file path
from shapely.geometry import Point
Point(-122.6, 49.3)       # a geometry, (lon, lat)

```

🌟 Challenge (optional — for a bit more)

Build a dictionary for a place **you** care about with keys `name`, `lat`, `lon`, and a list `neighbours` of 2–3 nearby coordinates. Then make a Shapely `LineString` through the neighbours and print its `.length`. (It's in degrees — we fix that in Chapter 5!)

🎓 Additional learning (optional)

If we have spare time we'll do these together — otherwise they're great self-study. Not required for the quiz or exercise, but they make the next chapters easier.

Bonus 1 — Formatting output with f-strings

Why code this? A result only matters if you can *communicate* it. An **f-string** drops variables into a string, so you can build clean report lines, map labels, and messages from your data — something you'll do in every chapter of this book.

```
species = "Douglas-fir"
dbh_cm = 47.5
plot_id = 12

# an f-string starts with f"..." — put variables inside { }
print(f"Plot {plot_id}: {species}, DBH {dbh_cm} cm")
print(f"That is about {dbh_cm / 2.54:.1f} inches.") # :.1f = round to 1
decimal
```

```
Plot 12: Douglas-fir, DBH 47.5 cm
That is about 18.7 inches.
```

🌲 Instead of printing bare numbers, foresters format results into readable summaries — e.g. a per-plot line in a stand report, or a label on a map.

Try it yourself. Write an f-string that prints `Site MKRF is at 49.27 N, -122.57 W` using variables.

▼  **Show answer**

```
lat, lon = 49.27, -122.57
print(f"Site MKRF is at {lat} N, {lon} W")
```

Bonus 2 — A forest inventory as a list of dictionaries

Why code this? Real datasets are *collections of records*. Modelling a forest inventory as a **list of dictionaries** — one dict per plot — is the exact mental model behind every table you'll meet in **Pandas** (Chapter 3) and **GeoPandas** (Chapter 4). Learn it here in miniature and those chapters feel familiar.

```
# each tree/plot is a dictionary; the inventory is a list of them
inventory = [
    {"plot": 1, "species": "Douglas-fir",      "dbh_cm": 47.5},
    {"plot": 2, "species": "western redcedar", "dbh_cm": 61.0},
    {"plot": 3, "species": "Douglas-fir",      "dbh_cm": 28.0},
]

# loop over the records
for tree in inventory:
    print(tree["plot"], "-", tree["species"], "-", tree["dbh_cm"], "cm")

# a quick summary: average DBH across the inventory
dbhs = [tree["dbh_cm"] for tree in inventory]
print("Average DBH:", round(sum(dbhs) / len(dbhs), 1), "cm")
```

```
1 - Douglas-fir - 47.5 cm
2 - western redcedar - 61.0 cm
3 - Douglas-fir - 28.0 cm
Average DBH: 45.5 cm
```

🌲 This is a forest inventory table, built from plain Python. In Chapter 3 Pandas will do the looping and summarising for you — but it's the same idea.

Try it yourself. Add a fourth tree to `inventory`, then count how many are "Douglas-fir" (hint: a loop with an `if`, or a list comprehension).

▼ Show answer

```
inventory.append({"plot": 4, "species": "Douglas-fir", "dbh_cm": 39.0})
n_df = sum(1 for tree in inventory if tree["species"] == "Douglas-fir")
print(n_df, "Douglas-fir trees")    # 3
```

Working with AI (30-second habit)

You may use AI assistants, but in this book you must be able to **explain and check** any code. Each chapter has a small "does this code do what it claims?" task. Habit to

start now: if AI gives you code, (1) read each line, (2) run it, (3) check the result is sensible, (4) confirm functions exist in the official docs.

Recap & what's next

Today: cells & comments · variables & types · operators · lists/tuples/dicts & indexing · file paths · importing packages · first Shapely geometries.

- **Practice now:** open the **Chapter 1 Exercise** (5 short parts, ~30–60 min).
- **Next chapter (Chapter 2):** programming logic & reusable code — conditions, loops, and functions, applied to coordinates and files.

Practice exercises

Practise variables, data structures, indexing, and a first Shapely geometry.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

1. Core

Create variables for a site: `name` (str), `lat` and `lon` (floats), `protected` (bool). Print a one-line summary.

```
# TODO – your code here
```

2. Apply

Make a list `coords` of three `(lon, lat)` tuples, and a dictionary `site` with keys `name`, `n_plots`, `region`.

```
# TODO – your code here
```

3. Interpret

Print the `type()` of `name`, `lat`, and `protected`. In a comment, note which is text, decimal, and true/false.

```
# TODO – your code here
```

4. Debug

This line should print the **latitude** of the first coordinate but is wrong. Fix it.

```
print(coords[1][0])
```

```
# TODO – your code here
```

5. Extension

Import Shapely and make a `Point` from the first coordinate; print it.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 1 exercise and want more.

You'll practise

1. Rewrite loops as **list/dict comprehensions**.
2. Format output neatly with **f-string specifiers**.
3. Write a small **function** with a default argument and a docstring.
4. Handle bad input safely with **try / except**.

1. Comprehensions — a loop on one line

A **list comprehension** builds a new list from an old one. It reads like English: *value for each item*.

```
heights_m = [30, 42, 51, 28, 47]
heights_ft = [round(h * 3.28084, 1) for h in heights_m]    # metres -> feet
print(heights_ft)

tall = [h for h in heights_m if h > 40]                    # keep only the tall
trees
print("tall (>40 m):", tall)
```

A **dict comprehension** does the same for dictionaries — here, counting species codes.

```
species = ["Df", "Cw", "Hw", "Df", "Cw"]
counts = {s: species.count(s) for s in set(species)}
print(counts)

codes = {"Df": "Douglas-fir", "Cw": "western redcedar", "Hw": "western
hemlock"}
print([codes[s] for s in species])
```

✓ Quick check — comprehensions

What does `[x*x for x in [1, 2, 3, 4] if x % 2 == 0]` produce?

▼  **Show answer**

`[4, 16]` — it keeps only the even numbers (2 and 4), then squares each.

2. f-string formatting — a tidy report

Inside an f-string, `:` starts a *format spec*: `<` left-aligns, `>` right-aligns, `.1f` fixes one decimal.

```
plots = [("A1", 47.5), ("A2", 61.0), ("B1", 28.0)]
for name, dbh in plots:
    print(f"{name:<4} DBH {dbh:>6.1f} cm")
```

3. Functions with a default and a docstring

A **default argument** lets a caller skip it. The `"""docstring"""` documents what the function does.

```
def dbh_class(dbh_cm, threshold=50):
    """Return 'large' if dbh_cm is above threshold, else 'small'."""
    return "large" if dbh_cm > threshold else "small"

print(dbh_class(61), dbh_class(28), dbh_class(45, threshold=40))
print(dbh_class.__doc__)
```

4. Safe input with try / except

Real field data has typos. `try/except` lets your code *keep going* instead of crashing.

```
def safe_lat(value):
    """Return a valid latitude float, or None if the value is bad."""
    try:
        v = float(value)
        return v if -90 <= v <= 90 else None
    except ValueError:
```

```
        return None

print(safe_lat("49.27"), safe_lat("oops"), safe_lat(120))
```

🌲 **Why code this?** Comprehensions and small functions are how you turn a one-off script into a **reusable tool** — the same `dbh_class` runs on 5 trees or 50,000, and `safe_lat` quietly cleans a messy inventory file instead of stopping at the first typo.

🌟 Challenge

Write a function `mean_dbh(records, species)` that takes a list of `(species, dbh)` tuples and returns the average DBH for one species (or `None` if that species is absent).

▼ 🌟 Show one solution

```
def mean_dbh(records, species):
    vals = [d for s, d in records if s == species]
    return round(sum(vals) / len(vals), 1) if vals else None

data = [("Df", 47.5), ("Cw", 61.0), ("Df", 28.0)]
print(mean_dbh(data, "Df"))    # 37.8
print(mean_dbh(data, "Hw"))    # None
```

🎓 Nicely done

You've gone a step beyond the Chapter 1 material. Next chapter we turn these functions into fully reusable, testable code.

Chapter 2 — Programming Logic & Reusable Code

Last chapter we stored data in variables, lists, and dictionaries. This chapter we make Python **decide** and **repeat** — conditions, loops, and **functions** — the tools for writing code you can reuse across many coordinates and files.

Learning outcomes

1. Use comparison (`>` `<` `==`) and logical (`and` `or` `not`) operators.
2. Write `if` / `elif` / `else` decisions.
3. Repeat work with `for` and `while` loops.
4. Write **functions** with parameters and a return value.
5. Import a module and read its documentation.
6. Read a traceback and handle errors with `try` / `except` . Kernel: **Python (gem530)**.

1. Comparison & logical operators

An operator like `>` returns a **boolean** (`True` / `False`). Logical operators combine booleans. Here we ask: *is a point inside a bounding box?*

```
lat, lon = 49.27, -122.57          # a point in MKRF
# bounding box (min/max) roughly around MKRF
lat_min, lat_max = 49.20, 49.35
lon_min, lon_max = -122.70, -122.45

inside = (lat_min <= lat <= lat_max) and (lon_min <= lon <= lon_max)
print("Inside the box?", inside)
```

🌲 **Why code this?** A single comparison like `dbh_cm > 50` becomes a *rule* you apply to a whole inventory — finding every large tree, or every stand steeper than

30°, without reading rows by hand.

2. Conditional statements (`if` / `elif` / `else`)

Do different things depending on a condition. Let's label a latitude by zone.

```
def_lat = 49.27
if def_lat > 66.5:
    zone = "Arctic"
elif def_lat > 23.5:
    zone = "Temperate"
elif def_lat > 0:
    zone = "Tropical (N)"
else:
    zone = "Southern hemisphere"
print(def_lat, "->", zone)
```

Try it yourself. Change `def_lat` to `-13.35` (the Madagascar site) and re-run. Which branch executes?

🌲 **Forestry example.** `if` / `elif` sorts data into categories — seedling / sapling / mature by DBH, or low / moderate / high fire risk by fuel load. The computer applies your rules the *same way* to thousands of records.

3. Loops

A **for** loop repeats once per item. A **while** loop repeats until a condition is false.

```
plot_coords = [(-122.573, 49.271), (-122.569, 49.268), (-122.566, 49.274)]

# for loop: visit every coordinate
for i, (x, y) in enumerate(plot_coords):
    print(f"Plot {i}: lon={x}, lat={y}")
```

```
# while loop: a simple countdown
n = 3
while n > 0:
    print("countdown", n)
    n = n - 1
```

🌲 **Why code this?** Surveys produce many plots and many files. A loop runs the same step on every one — score 500 plots, read 200 rasters — from code you write once.

✓ Quick check — loops

How many times does the body run? `for x in [3, 6, 9]:`

▼ ► Show answer

3 — once per item in the list. The values are 3, 6, 9.

4. Functions: reusable code

A **function** packages steps under a name so you can reuse them. It takes **parameters** and can **return** a value. This is the most important idea this chapter.

```
def is_valid_coord(lat, lon):
    """Return True if lat/lon are within valid Earth ranges."""
    return (-90 <= lat <= 90) and (-180 <= lon <= 180)

print(is_valid_coord(49.27, -122.57))    # True
print(is_valid_coord(120.0, -122.57))    # False: latitude out of range
```

```
def bounding_box(coords):
    """Given a list of (lon, lat), return (min_lon, min_lat, max_lon,
    max_lat)."""
    lons = [c[0] for c in coords]
    lats = [c[1] for c in coords]
    return (min(lons), min(lats), max(lons), max(lats))
```

```
print("BBox:", bounding_box(plot_coords))
```

Try it yourself. Write a function `midpoint(a, b)` that takes two `(lon, lat)` tuples and returns the average point. Test it on two plots.

🌲 **Why code this?** A function captures a calculation (e.g. basal area from DBH) so you compute it *identically everywhere* and fix it in one place. Reproducibility starts here.

✅ Quick check — return vs print

A function ends with `return a + b`, but you see nothing on screen when you call it. Why?

▼ ► Show answer

`return` hands the value *back* to the caller; it does not print. Do `print(add(2, 3))` or store it: `total = add(2, 3)`.

5. Modules & reading documentation

`import` a module, then use its tools. Use `help()` (or the official docs) to learn what a function expects.

```
import statistics
lats = [c[1] for c in plot_coords]
print("Mean latitude:", round(statistics.mean(lats), 5))
# help(statistics.mean)  # uncomment to read the documentation
```

🌲 **Forestry example.** The `statistics` module summarises field measurements; later, geospatial modules do the heavy lifting. Reading the docs is how you pick up a new tool on the job.

6. Errors & debugging

Errors are normal. Read the **last line** of a traceback first — it names the error type. Use `try` / `except` to handle problems gracefully.

```
from pathlib import Path
try:
    text = Path("does_not_exist.csv").read_text()
except FileNotFoundError:
    print("That file is missing – check the path (see Chapter 5).")
```

Check inputs before using them — a small guard prevents confusing errors later:

```
def safe_report(lat, lon):
    if not is_valid_coord(lat, lon):
        return "Rejected: coordinate out of range"
    return f"OK: {lat}, {lon}"

print(safe_report(49.27, -122.57))
print(safe_report(999, 0))
```

🌲 **Why code this?** Field data is messy — missing files, impossible values. `try` / `except` and input checks keep a batch job running instead of crashing on plot #347.

🔧 Hands-on activity — Classify the stand (~20 min)

Write a function that assigns a **size class** to a tree by its DBH, then run it over a small inventory and count the mature trees. Work individually.

Task 1. Write `size_class(dbh)` returning "seedling" (<2.5 cm), "sapling" (2.5–10 cm), or "mature" (>10 cm). **Task 2.** Loop over the `inventory` below and print each tree's class. **Task 3.** Count how many trees are "mature".

```
inventory = [1.2, 8.0, 15.5, 42.0, 3.0, 27.5]    # DBH in cm

# TODO Task 1: def size_class(dbh): ...
# TODO Task 2: loop and print each class
# TODO Task 3: count the "mature" trees
```

Activity solutions

▼ ► Task 1 — the function

```
def size_class(dbh):
    if dbh < 2.5:
        return "seedling"
    elif dbh <= 10:
        return "sapling"
    return "mature"
```

▼ ► Task 2 — loop & print

```
for dbh in inventory:
    print(dbh, "cm ->", size_class(dbh))
```

▼ ► Task 3 — count mature

```
n_mature = sum(1 for dbh in inventory if size_class(dbh) == "mature")
print("mature trees:", n_mature)
```

Common errors this chapter (and the fix)

Message / symptom	Likely cause	Fix
loop runs forever	a <code>while</code> condition never becomes False	change something inside the loop
<code>IndentationError</code>	body of <code>if</code> / <code>for</code> / <code>def</code> not indented	indent 4 spaces consistently

Message / symptom	Likely cause	Fix
function "returns nothing"	used <code>print</code> instead of <code>return</code> , or forgot <code>return</code>	<code>return</code> the value, then print the call
<code>TypeError</code> in a calc	mixing text and numbers (e.g. from a file)	convert first: <code>float(x)</code>

Quick reference — Chapter 2 cheat sheet

```

if dbh > 50:           # condition
    ...
elif dbh > 10:
    ...
else:
    ...

for x in items:       # loop over a collection
    ...

def basal_area(dbh):  # define a reusable function
    return 3.14159 * (dbh/2)**2

try:
    risky()           # code that might fail
except FileNotFoundError:
    ...               # handle it gracefully

```

Additional learning (optional)

If we have time we'll do these together — otherwise great self-study. Not required for the quiz or exercise.

Bonus 1 — List comprehensions (loops in one line)

Why code this? Filtering and transforming lists is constant in data work; a **list comprehension** does it in one readable line — you'll use it to pull columns, filter plots, and build inputs for maps.

```
dbhs = [1.2, 8.0, 15.5, 42.0, 3.0, 27.5]

# keep only the big trees (DBH > 10) – one line instead of a loop
big = [d for d in dbhs if d > 10]
print("big trees:", big)

# transform: DBH in cm -> radius in cm
radii = [d / 2 for d in dbhs]
print("radii:", radii)
```

🌲 Same idea you'd use to grab every plot above a threshold, or convert units for a whole column.

Try it yourself. Make a list of only the "mature" DBHs (> 10) rounded to whole numbers.

Bonus 2 — Pairing two lists with `zip`

Why code this? Measurements often arrive in *parallel lists* (species here, DBH there). `zip` pairs them up so you can build records — the bridge to the tables you'll meet in Pandas (Chapter 3).

```
species = ["Douglas-fir", "redcedar", "hemlock"]
dbhs     = [47.5, 61.0, 28.0]

for sp, d in zip(species, dbhs):
    print(f"{sp}: {d} cm")

# build a list of dictionaries (a mini inventory table)
records = [{"species": sp, "dbh_cm": d} for sp, d in zip(species, dbhs)]
print(records)
```

🌲 This `records` list is exactly the shape Pandas turns into a DataFrame next chapter.

Packages used this chapter

- **Python standard library** — the core language: operators, `if`, loops, `def`.
- **statistics** (standard library) — quick summaries like `mean`, `median`.
Geospatial link: summarize coordinate lists or attribute values before mapping.
- **pathlib** — safe file paths; here used to trigger and catch a file error.

See the [Package Guide](#).

Working with AI (30-second habit)

If AI writes a loop or function for you: read it line by line, run it on a **small** input you can check by hand, and confirm the returned value is what you expect. Your Chapter 2 exercise includes one "fix this broken function" task.

Recap & what's next

Operators · `if/elif/else` · `for` / `while` loops · **functions** · modules · errors & `try/except`.

- **Practice:** Chapter 2 Exercise (~30–60 min).
- **Next chapter:** tabular data with **Pandas** — the DataFrame.

Practice exercises

Practise conditionals, loops, and writing reusable functions.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

Setup (run first)

```
lats = [49.27, -13.35, 0.0, 91.0]
```

1. Core

Write a function `hemisphere(lat)` that returns `"N"` for positive latitude, `"S"` for negative, and `"equator"` for 0.

```
# TODO – your code here
```

2. Apply

Loop over `lats` and print each latitude with its hemisphere.

```
# TODO – your code here
```

3. Interpret

What does `hemisphere(0)` return, and why? Answer in a comment.

```
# TODO – your code here
```

4. Debug

This function should count valid latitudes ($-90..90$) but always returns 0. Fix it.

```
def count_valid(values):
    n = 0
    for v in values:
        if -90 <= v <= 90:
            n = 0 + 1
    return n
```

```
# TODO – your code here
```

5. Extension

Add a docstring and a second parameter `strict=False` to `hemisphere` that, when `True`, raises `ValueError` for an out-of-range latitude.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 2 exercise and want more.

You'll practise

1. Write functions that **return** values (and several at once).
2. Combine lists with **zip** and **enumerate**.
3. Guard against empty input with an early **return**.
4. Assemble a few functions into a reusable **toolkit**.

1. Functions that return values

A good function *returns* a result you can reuse, rather than only printing it.

```
def basal_area(dbh_cm):
    """Basal area (m^2) of a tree from its DBH in cm."""
    radius_m = (dbh_cm / 100) / 2
    return round(3.14159 * radius_m ** 2, 4)

dbhs = [30, 42, 51]
areas = [basal_area(d) for d in dbhs]
print(areas)
print("stand basal area:", round(sum(areas), 4), "m^2")
```

2. zip and enumerate

`zip` walks two lists together; `enumerate` gives you a counter for free.

```
species = ["Df", "Cw", "Hw"]
dbhs     = [47.5, 61.0, 28.0]
for i, (sp, d) in enumerate(zip(species, dbhs), start=1):
    print(f"{i}. {sp}: {d} cm")
```

✓ Quick check — zip

How many items does `list(zip([1, 2, 3], ['a', 'b']))` contain?

▼  **Show answer**

2 — `zip` stops at the shorter list, giving `[(1, 'a'), (2, 'b')]`.

3. Multiple return values + a guard clause

A **guard clause** handles the empty case first, so the rest of the function can assume good input.

```
def summarize(values):
    """Return (min, max, mean) — or (None, None, None) if empty."""
    if not values:
        return None, None, None
    return min(values), max(values), round(sum(values) / len(values), 1)

lo, hi, mean = summarize(dbhs)
print("min", lo, "| max", hi, "| mean", mean)
print(summarize([]))
```

4. A tiny reusable toolkit

Group related functions together — this is the seed of your own analysis library.

```
def to_feet(m):      return round(m * 3.28084, 1)
def is_big(dbh, k=50): return dbh > k
def classify(dbh):    return "large" if is_big(dbh) else "small"

print(to_feet(40), is_big(61), classify(28))
print([classify(d) for d in dbhs])
```

🌲 **Why code this?** Every forestry model — volume, biomass, carbon — is just a stack of small functions like these. Writing them once, with clear names and returns, means your Chapter-11 machine-learning features and your Lab analyses can all call the *same tested code*.

🌟 Challenge

Write `big_trees(records, k=50)` that returns the list of species whose DBH exceeds `k`, from a list of `(species, dbh)` tuples.

▼ 🌟 Show one solution

```
def big_trees(records, k=50):  
    return [s for s, d in records if d > k]  
  
data = [("Df", 47.5), ("Cw", 61.0), ("Hw", 28.0)]  
print(big_trees(data))          # ['Cw']  
print(big_trees(data, k=25))    # ['Df', 'Cw', 'Hw']
```

🎓 Nicely done

You've gone a step beyond the Chapter 2 material. You now have the tools to write clean, reusable analysis code throughout.

Chapter 3 — Tabular Data with Pandas

Most environmental data arrives as **tables** — plots, fire records, field measurements. **Pandas** is the tool for loading, cleaning, filtering, and summarizing tables in Python. Its main object is the **DataFrame**.

Learning outcomes

1. Create/read a **DataFrame** and inspect it (`head` , `info` , `describe`).
2. **Select**, **filter**, and **sort** rows and columns.
3. Add a **computed column** and handle **missing values**.
4. **Group** and **summarize** with `groupby` .
5. **Join** two tables with `merge` .
6. Sanity-check coordinate columns and **export** cleaned data. Kernel: **Python (gem530)**.

1. A DataFrame

A DataFrame is a table: named **columns**, numbered **rows**. Here is a small fire-occurrence table (like Canada's National Fire Database).

```
import pandas as pd

data = {
    "site":  ["MKRF", "MKRF", "Ambaro", "Ambaro", "MKRF", "Ambaro"],
    "year":  [2019, 2020, 2019, 2021, 2021, 2020],
    "lat":   [49.27, 49.29, -13.35, -13.40, 49.31, -13.33],
    "lon":   [-122.57, -122.55, 48.55, 48.60, -122.52, 48.58],
    "area_ha": [12.5, 3.0, 40.0, None, 7.5, 22.0],    # note the missing value
    "cause":  ["lightning", "human", "human", "human", "lightning",
"lightning"],
}
df = pd.DataFrame(data)
df.head()
```

```
df.info()      # column names, types, and non-null counts
df.describe()  # quick numeric summary
```

🌲 **Why code this?** A DataFrame is a forest-inventory table *in code* — plots as rows, measurements as columns — that you can filter, summarise, and join far faster and more reliably than a spreadsheet.

2. Select, filter, sort

```
# select columns
print(df[["site", "area_ha"]].head())

# filter rows: only lightning fires
lightning = df[df["cause"] == "lightning"]
print("\nLightning fires:")
print(lightning)

# sort by area, largest first
print("\nSorted by area:")
print(df.sort_values("area_ha", ascending=False).head())
```

🌲 **Forestry example.** Filtering pulls out just the fir plots, or fires over 100 ha; sorting ranks stands by volume. One line replaces scrolling and highlighting cells.

✓ Quick check — filtering

Which expression keeps only the rows where `species` is `"fir"`?

▼ ► Show answer

`df[df["species"] == "fir"]` — a boolean filter keeps the rows where the condition is True.

3. New columns & missing values

`area_ha` has a missing value (`NaN`). We can fill or drop it, and we can compute a new column.

```
# a computed column: area in square kilometres
df["area_km2"] = df["area_ha"] / 100.0

# how many missing?
print("Missing area_ha:", df["area_ha"].isna().sum())

# fill missing area with 0 (a choice you must justify!)
df["area_ha"] = df["area_ha"].fillna(0)
df[["site", "year", "area_ha", "area_km2"]]
```

🌲 **Why code this?** Derived columns (basal area, biomass, area in ha) and honest handling of missing measurements are everyday inventory tasks — and how you avoid silently wrong totals.

4. Group & summarize

`groupby` answers "for each group, what is ...?" — a core analysis move.

```
# total burned area per site
by_site = df.groupby("site")["area_ha"].sum()
print(by_site)

# count of fires per cause
print("\nFires per cause:")
print(df.groupby("cause").size())
```

🌲 **Why code this?** `groupby` answers the questions managers ask: total burned area per region, mean DBH per species, plots per treatment — the heart of a summary report.

✅ Quick check — `groupby`

In words, what does `df.groupby("region")["area_ha"].sum()` give you?

▼ ► **Show answer**

The **total area_ha for each region** — one summed value per group. Swap `.sum()` for `.mean()` to get an average.

5. Join two tables

Real data is split across tables. `merge` combines them on a shared key. Here we add a region label per site.

```
regions = pd.DataFrame({
    "site":  ["MKRF", "Ambaro"],
    "region": ["British Columbia, Canada", "NW Madagascar"],
})
df = df.merge(regions, on="site", how="left")
df[["site", "region", "year", "area_ha"]].head()
```

🌲 **Forestry example.** Field data and lookup tables live apart — plot measurements here, species codes or region names there. `merge` stitches them so every record carries the labels it needs.

6. Check coordinates & export

Before mapping (Chapter 4), confirm latitude/longitude are in valid ranges, then export the cleaned table.

```
bad = df[(df["lat"].abs() > 90) | (df["lon"].abs() > 180)]
print("Rows with impossible coordinates:", len(bad))

from pathlib import Path
Path("data").mkdir(exist_ok=True)
out = Path("data") / "fires_clean.csv"
df.to_csv(out, index=False)
print("Wrote", out)
```

```
# read it back to prove the round-trip works
again = pd.read_csv(out)
print("Read back", len(again), "rows")
```

Try it yourself. Compute the **average** `area_ha` per `year` using `groupby`. Which year burned most on average?

🌲 **Why code this?** A quick coordinate sanity check catches swapped lat/lon *before* they wreck a map, and exporting a clean table makes your analysis shareable and reproducible.

🔧 Hands-on activity — Summarise a plot inventory (~20 min)

Build a tiny inventory, add a derived column, and summarise it. Work individually.

Task 1. From the dict below, make a DataFrame and show it. **Task 2.** Add a column `basal_area` = `3.14159 * (dbh_cm/2)**2`. **Task 3.** Report the **mean** `dbh_cm` per **species**.

```
import pandas as pd
data = {
    "plot":    [1, 2, 3, 4],
    "species": ["fir", "cedar", "fir", "hemlock"],
    "dbh_cm":  [47.5, 61.0, 28.0, 33.0],
}
# TODO Task 1: build the DataFrame and show it
# TODO Task 2: add basal_area
# TODO Task 3: mean dbh_cm per species
```

Activity solutions

▼ ► Task 1 — DataFrame

```
df = pd.DataFrame(data)
df
```

▼ ► Task 2 — derived column

```
df["basal_area"] = 3.14159 * (df["dbh_cm"] / 2) ** 2
df
```

▼ ► Task 3 — group summary

```
print(df.groupby("species")["dbh_cm"].mean())
```

Common errors this chapter (and the fix)

Message / symptom	Likely cause	Fix
KeyError: 'species'	column name typo	check <code>df.columns</code>
merge gives too few/many rows	wrong <code>on=</code> key or duplicates	verify the key column matches both tables
a sum ignores some rows	missing values (NaN)	decide: <code>fillna(...)</code> or <code>dropna(...)</code> and say so
SettingWithCopyWarning	assigning into a filtered slice	build the column on the full frame, or use <code>.copy()</code>

Quick reference — Chapter 3 cheat sheet

```
import pandas as pd
df = pd.read_csv("data/plots.csv") # load a table
df.head(); df.info(); df.describe() # look at it
df[df["dbh_cm"] > 40] # filter rows
df["ba"] = 3.14159*(df["dbh_cm"]/2)**2 # new column
df.groupby("species")["dbh_cm"].mean() # summarise per group
```

```
df.merge(lookup, on="species")      # join tables
df.to_csv("out.csv", index=False)   # export
```

Additional learning (optional)

Great if we have time, otherwise self-study. Not required for the quiz or exercise.

Bonus 1 — Fast exploring: `describe()` and `value_counts()`

Why code this? Before analysing, you *look*. Two one-liners give you the numeric summary and the category counts of any inventory — the first thing a data analyst runs.

```
import pandas as pd
df = pd.DataFrame({
    "species": ["fir", "cedar", "fir", "hemlock", "fir", "cedar"],
    "dbh_cm":  [47.5, 61.0, 28.0, 33.0, 52.0, 40.0],
})
print(df["dbh_cm"].describe())      # count, mean, min, max, quartiles
print()
print(df["species"].value_counts()) # how many plots of each species
```

 `value_counts()` instantly tells you the species composition of a stand.

Bonus 2 — Cross-tabs with `pivot_table`

Why code this? Managers love a table of *X by Y* — mean DBH by species **and** region. `pivot_table` builds it in one call.

```
import pandas as pd
df = pd.DataFrame({
    "region":  ["MKRF", "MKRF", "Ambaro", "Ambaro", "MKRF"],
    "species": ["fir", "cedar", "mangrove", "mangrove", "fir"],
    "dbh_cm":  [47.5, 61.0, 20.0, 24.0, 52.0],
})
table = df.pivot_table(values="dbh_cm", index="species",
```

```
columns="region", aggfunc="mean")  
print(table)
```

🌲 This is the summary table you'd drop straight into a stand or monitoring report.

Packages used this chapter

- **Pandas** — the DataFrame: load, clean, filter, group, join, and export tables. *Why:* most field/attribute data is tabular. *When:* any time data starts as a CSV/Excel/table. *Geospatial link:* the attribute side of spatial data, and the lat/lon table we turn into map points next chapter. See the [Package Guide](#).

Working with AI (30-second habit)

AI is good at Pandas one-liners — but check the result's **shape** and a few rows. A wrong `merge` key or `groupby` column silently gives a plausible-looking but wrong answer. Your exercise includes one merge to debug.

Recap & what's next

DataFrame · select/filter/sort · new columns & missing values · `groupby` · `merge` · coordinate checks · export.

- **Practice:** Chapter 3 Exercise.
- **Next chapter:** turn this table into map features with **GeoPandas**.

Practice exercises

Practise loading, filtering, grouping, joining, and exporting a table.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

Setup (run first)

```
import pandas as pd
df = pd.DataFrame({
    "plot": ["p1", "p2", "p3", "p4", "p5", "p6"],
    "species": ["fir", "fir", "cedar", "cedar", "hemlock", "fir"],
    "dbh_cm": [30, 45, 52, 28, None, 39],
    "region": ["MKRF", "MKRF", "MKRF", "Ambaro", "Ambaro", "Ambaro"],
})
```

1. Core

Show only the rows where `species == "fir"`.

```
# TODO – your code here
```

2. Apply

Compute the **mean dbh_cm per species** (missing values are ignored by `mean`).

```
# TODO – your code here
```

3. Interpret

Which species has the largest mean DBH? State it in a comment.

```
# TODO – your code here
```

4. Debug

This join uses the wrong key and fails. Fix it so the region owner is added.

```
owners = pd.DataFrame({"region": ["MKRF", "Ambaro"], "owner":  
["UBC", "Community"]})  
df.merge(owners, on="site")
```

```
# TODO – your code here
```

5. Extension

Add a column `dbh_class` = "large" if `dbh_cm` $\geq$ 40 else "small" (leave missing as NaN), then export `df` to `outputs/plots.csv`.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 3 exercise and want more.

You'll practise

1. Summarise groups with **groupby().agg()** and named aggregations.
2. Reshape data with **pivot_table**.
3. Combine tables with **merge** (left vs inner).
4. Handle missing values with **fillna** / **dropna** and method chaining.

1. A small fires table

We'll reuse the earlier fire records (MKRF + Madagascar) as a DataFrame.

```
import pandas as pd, numpy as np
fires = pd.DataFrame({
    "site":  ["MKRF", "MKRF", "Ambaro", "Ambaro", "MKRF"],
    "year":  [2019, 2020, 2019, 2020, 2021],
    "area_ha": [12.5, 3.0, 40.0, 22.0, 7.5],
    "cause": ["lightning", "human", "human", "lightning", "lightning"],
})
print(fires)
```

2. groupby with named aggregations

Name each output column and the function that fills it — clear and self-documenting.

```
g = fires.groupby("site").agg(
    n=("area_ha", "size"),
    total_ha=("area_ha", "sum"),
    mean_ha=("area_ha", "mean"),
)
print(g)
```

3. pivot_table — sites × causes

`pivot_table` cross-tabulates: one variable down the rows, another across the columns.

```
pt = fires.pivot_table(index="site", columns="cause", values="area_ha",
                        aggfunc="sum", fill_value=0)
print(pt)
```

✓ Quick check — groupby

In `df.groupby("site").agg(n=("area_ha", "size"))`, what does `n` count?

▼ ✓ Show answer

The number of **rows in each site group** (fires per site) — `size` counts rows, including any with missing area.

4. merge — attach a lookup table

`how="left"` keeps every fire row and adds the manager where it matches.

```
owners = pd.DataFrame({"site": ["MKRF", "Ambaro"], "manager": ["UBC",
"Community"]})
merged = fires.merge(owners, on="site", how="left")
print(merged[["site", "year", "area_ha", "manager"]])
```

5. Missing data + method chaining

Chain operations to read top-to-bottom. `.assign` adds a column without mutating the original.

```
df = fires.copy()
df.loc[1, "area_ha"] = np.nan
print("missing area_ha:", df["area_ha"].isna().sum())
```

```
filled = df.assign(area_ha=df["area_ha"].fillna(df["area_ha"].median()))
print("after fill:", filled["area_ha"].tolist())
```

🌲 **Why code this?** Field campaigns produce messy tables: gaps, duplicate plots, mixed units. `groupby`, `pivot_table` and `merge` are the daily tools that turn raw plot records into the summaries a forest manager actually reads — area burned per district, mean DBH per stand, volume per owner.

🌟 Challenge

From `fires`, compute the **mean area per cause per year** as a table with years down the rows and causes across the columns (fill blanks with 0).

▼ 🌟 Show one solution

```
out = fires.pivot_table(index="year", columns="cause",
                        values="area_ha", aggfunc="mean", fill_value=0)
print(out)
```

🎓 Nicely done

You've gone a step beyond the Chapter 3 material. These reshaping moves carry straight into GeoPandas, where rows also carry geometry.

Chapter 4 — Vector Data with GeoPandas & Shapely

Last chapter's table had `lat / lon` columns. **GeoPandas** turns a table into a **GeoDataFrame** — a DataFrame with a `geometry` column of **Shapely** shapes — so we can filter, measure, join, map, and export real vector data.

Run this notebook in the **Python (gem530)** environment (GeoPandas installed).

Learning outcomes

1. Build a **GeoDataFrame** from coordinates and read a vector file.
2. Inspect `geometry`, attributes, and the **CRS**.
3. Filter by attribute; compute area / length / centroid.
4. Do one **attribute join** and one simple **spatial join**.
5. **Export** to a GeoPackage. This is about data **handling** — not advanced spatial analysis (that's the R course).

1. From a table to points

We reuse Chapter 3's cleaned fires table and make a point per row. GeoPandas uses `(x, y) = (lon, lat)` and we tag the CRS as WGS84 (`EPSG:4326`).

```
import pandas as pd
import geopandas as gpd

# In practice you would read your Chapter 3 output: df =
pd.read_csv("data/fires_clean.csv")
# Here we build a small sample so this notebook runs on its own:
df = pd.DataFrame({
    "site":  ["MKRF", "MKRF", "Ambaro", "Ambaro"],
    "cause": ["lightning", "human", "human", "lightning"],
    "lon":   [-122.57, -122.55, 48.55, 48.60],
```

```

    "lat":    [49.27, 49.29, -13.35, -13.40],
    "area_ha": [12.5, 3.0, 40.0, 22.0],
})

gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(df["lon"], df["lat"]),
    crs="EPSG:4326",
)
print(type(gdf))
print("CRS:", gdf.crs)
gdf.head()

```

🌲 **Why code this?** Your field GPS gives lon/lat columns; turning them into point geometries is the step that lets you map plots, measure distances, and ask *'what falls inside the reserve?'*

✅ Quick check — point order

To build points from a table, is it `points_from_xy(lon, lat)` or `points_from_xy(lat, lon)`?

▼ ► Show answer

`points_from_xy(lon, lat)` — **x = longitude first**. Reversing them puts every plot in the wrong place.

2. Read a vector file

Real layers come as shapefiles, GeoPackages, or geodatabases. `read_file` loads any of them into a `GeoDataFrame`.

```

# Example (adjust the path to the provided data):
# stands = gpd.read_file("data/MKRF/stands.shp")
# print(stands.crs)
# stands.head()

```

🌲 **Forestry example.** Stand boundaries, road networks, stream layers, protected areas — all arrive as shapefiles or GeoPackages. `read_file` loads any of them into one consistent table you can analyse.

3. Inspect geometry, attributes, CRS, and a quick map

```
print("Columns:", list(gdf.columns))
print("Geometry type:", gdf.geom_type.unique())
gdf.plot(figsize=(5, 5), markersize=30) # a quick look
```

🌲 **Why code this?** Checking the CRS and geometry type up front prevents the classic disasters — points in the ocean, areas of zero — before they reach a map or a report.

4. Filter by attribute

```
mkrf = gdf[gdf["site"] == "MKRF"]
print("MKRF fire points:", len(mkrf))
mkrf.head()
```

🌲 **Forestry example.** Filtering a layer to one management unit, one species, or one disturbance year is the everyday first move before mapping or summarising.

5. Geometry measurements

`area`, `length`, and `centroid` are computed on the geometry. **Important:** in a geographic CRS (degrees) these are *not* real metres — we fix that in **Chapter 5** by reprojecting. For polygons you would use `gdf.area`; for now we show centroids.

```
# centroid of each geometry (for points, the point itself)
gdf["centroid"] = gdf.geometry.centroid
```

```
print(gdf[["site", "centroid"]].head())
```

🌲 **Why code this?** Area of a cutblock, length of a stream buffer, the centroid of a stand — these drive planning. (Compute them in a *projected* CRS — that's Chapter 5.)

✅ Quick check — area units

A GeoDataFrame is still in EPSG:4326 (degrees). Is `gdf.area` a real area in square metres?

▼ ► Show answer

No — in a geographic CRS the number is in *square degrees*, which is meaningless. Reproject to a projected CRS first (Chapter 5).

6. Attribute join + one spatial join

An **attribute join** matches on a shared column (like Chapter 3's `merge`). A **spatial join** matches by *location* — e.g. which region polygon each point falls in.

```
# attribute join: add a lookup column
info = pd.DataFrame({"site": ["MKRF", "Ambaro"], "manager": ["UBC",
"Community"]})
gdf = gdf.merge(info, on="site", how="left")

# spatial join (example – needs a polygon layer 'regions'):
# regions = gpd.read_file("data/regions.gpkg").to_crs(gdf.crs)
# joined = gpd.sjoin(gdf, regions, how="left", predicate="within")
# joined.head()
gdf[["site", "manager"]].head()
```

🌲 **Forestry example.** Joins answer '*which plots fall in which watershed?*' (spatial) and '*attach each stand's manager*' (attribute) — combining field data with reference layers.

7. Export

Save results as a GeoPackage (`.gpkg`) — a modern single-file format that keeps the CRS and attributes.

```
from pathlib import Path
Path("data").mkdir(exist_ok=True)
gdf.drop(columns=["centroid"]).to_file("data/fires_points.gpkg", driver="GPKG")
print("Wrote data/fires_points.gpkg")
```

Try it yourself. Filter `gdf` to the **Ambaro** site and plot only those points. How many are there?

🌲 **Why code this?** Exporting a clean GeoPackage means your result opens in QGIS/ArcGIS and re-runs next season — reproducible, shareable geospatial work.

🔧 Hands-on activity — Map your plots (~20 min)

Turn a small plot table into a mapped GeoDataFrame. Work individually.

Task 1. Make a GeoDataFrame of points from the table below (`crs="EPSG:4326"`).

Task 2. Filter to the `"MKRF"` region and count the plots. **Task 3.** Plot the points coloured by `region`.

```
import geopandas as gpd, pandas as pd
data = {
    "plot":    [1, 2, 3, 4, 5],
    "region":  ["MKRF", "MKRF", "Ambaro", "Ambaro", "MKRF"],
    "lon":     [-122.57, -122.55, 48.55, 48.60, -122.52],
    "lat":     [49.27, 49.29, -13.35, -13.40, 49.31],
}
# TODO Task 1: build the GeoDataFrame (points_from_xy, crs="EPSG:4326")
# TODO Task 2: filter to region == "MKRF" and count
# TODO Task 3: plot coloured by region
```

Activity solutions

▼ ► Task 1 — points

```
df = pd.DataFrame(data)
gdf = gpd.GeoDataFrame(
    df, geometry=gpd.points_from_xy(df["lon"], df["lat"]), crs="EPSG:4326")
gdf
```

▼ ► Task 2 — filter & count

```
mkrf = gdf[gdf["region"] == "MKRF"]
print(len(mkrf), "MKRF plots")
```

▼ ► Task 3 — plot

```
gdf.plot(column="region", legend=True, markersize=40)
```

Common errors this chapter (and the fix)

Message / symptom	Likely cause	Fix
points in the wrong place	lon/lat reversed	<code>points_from_xy(lon, lat)</code> — x first
<code>.crs</code> is <code>None</code>	forgot to set the CRS	pass <code>crs="EPSG:4326"</code> when building
spatial join is empty	layers in different CRSs	<code>other = other.to_crs(gdf.crs)</code> first
area/length is tiny/odd	data still in degrees	reproject to a projected CRS (Chapter 5)

Quick reference — Chapter 4 cheat sheet

```
import geopandas as gpd
gdf = gpd.read_file("data/stands.gpkg")          # read vector
gdf = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df.lon, df.lat),
```

```

crs="EPSG:4326")
gdf.crs; gdf.geom_type; gdf.plot()           # inspect
gdf[gdf["species"] == "fir"]                 # filter
gdf.geometry.area; gdf.geometry.length      # measure (projected CRS!)
gpd.sjoin(gdf, regions, predicate="within")  # spatial join
gdf.to_file("out.gpkg", driver="GPKG")       # export

```

Additional learning (optional)

Great if we have time, otherwise self-study. Not required for the quiz or exercise.

Bonus 1 — Buffers (a riparian zone around a stream)

Why code this? A **buffer** is a zone a fixed distance from a feature — a 30 m riparian reserve along a stream, a setback around a road. It's one of the most-used tools in forest planning. (Buffer in a *projected* CRS so the distance is real metres.)

```

import geopandas as gpd
from shapely.geometry import LineString
stream = gpd.GeoDataFrame(geometry=[LineString([(-122.60, 49.30), (-122.55,
49.32)])],
                           crs="EPSG:4326")
stream_m = stream.to_crs("EPSG:3005")      # BC Albers -> metres
riparian = stream_m.buffer(30)              # 30-metre buffer
print("riparian buffer area (m^2):", round(riparian.area.iloc[0], 1))

```

 Change 30 to your reserve width — instant riparian zones for any stream layer.

Bonus 2 — "What's inside the reserve?" with a spatial test

Why code this? Deciding which plots fall inside a boundary is a core GIS question — `within` answers it for every feature at once, no clicking.

```

import geopandas as gpd
from shapely.geometry import Point, box
plots = gpd.GeoDataFrame(geometry=[Point(-122.57, 49.27), Point(-121.0, 49.0)],

```

```
crs="EPSG:4326")
reserve = box(-122.75, 49.20, -122.45, 49.40)    # a reserve boundary
inside = plots[plots.within(reserve)]
print("plots inside the reserve:", len(inside), "of", len(plots))
```

🌲 Swap the box for a real protected-area polygon to audit which plots it contains.

Packages used this chapter

- **GeoPandas** — the vector workhorse: read/write layers, filter, measure, join, and map, all with a `geometry` column. *Geospatial link*: the core tool for vector data handling in Python.
- **Shapely** — the geometry objects (Point/LineString/Polygon) inside each GeoDataFrame row; you met it in Chapter 1. See the [Package Guide](#).

Working with AI (30-second habit)

GeoPandas errors often come from a **missing or mismatched CRS**. If AI code fails, check `gdf.crs` first, and confirm both layers share a CRS before any spatial join. More on CRS next chapter.

Recap & what's next

GeoDataFrame · points from a table · read vector files · inspect CRS/geometry · filter · measure · attribute + spatial joins · export GeoPackage.

- **Practice:** Chapter 4 Exercise.
- **Next chapter:** coordinate reference systems, files & data quality (Lab 1 prep).

Practice exercises

Practise building, inspecting, filtering, and exporting vector data.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

Run in **Python (gem530)** (GeoPandas installed).

Setup (run first)

```
import pandas as pd, geopandas as gpd
df = pd.DataFrame({
    "site": ["a", "b", "c", "d"],
    "region": ["MKRF", "MKRF", "Ambaro", "Ambaro"],
    "lon": [-122.57, -122.55, 48.55, 48.60],
    "lat": [49.27, 49.29, -13.35, -13.40],
})
```

1. Core

Build a GeoDataFrame `gdf` of points from `lon / lat` with `crs="EPSG:4326"`.

```
# TODO – your code here
```

2. Apply

Print the geometry type and make a quick `plot()`.

```
# TODO – your code here
```

3. Interpret

Filter `gdf` to the MKRF region and report how many points it has (comment on the count).

```
# TODO – your code here
```

4. Debug

This build has no CRS, so later reprojection fails. Fix it.

```
gdf2 = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df.lon, df.lat))
```

```
# TODO – your code here
```

5. Extension

Export `gdf` to `outputs/points.gpkg` (GeoPackage).

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 4 exercise and want more.

You'll practise

1. Find which points fall in which polygon with **sjoin**.
2. Build **buffers** and measure their area.
3. Merge features by attribute with **dissolve**.
4. Measure **distances** between geometries.

1. Plots and a reserve

We work in a **projected CRS (BC Albers, EPSG:3005)** so distances and areas are in metres.

```
import geopandas as gpd
from shapely.geometry import Point, Polygon
plots = gpd.GeoDataFrame(
    {"plot": ["p1", "p2", "p3", "p4"], "species": ["Df", "Cw", "Hw", "Df"]},
    geometry=[Point(0, 0), Point(5, 5), Point(20, 20), Point(2, 1)],
    crs="EPSG:3005")
reserve = gpd.GeoDataFrame(
    {"name": ["reserve"]},
    geometry=[Polygon([(-1, -1), (10, -1), (10, 10), (-1, 10)])],
    crs="EPSG:3005")
print(plots)
```

2. Spatial join — which plots are in the reserve?

`sjoin` with `predicate="within"` keeps points that fall inside a polygon.

```
inside = gpd.sjoin(plots, reserve, predicate="within", how="inner")
print(inside[["plot", "species", "name"]])
print("plots inside:", len(inside), "of", len(plots))
```

✓ Quick check — predicates

Which predicate keeps points that fall *inside* a polygon: `within` or `contains`?

▼ ✓ Show answer

`within` — a point is *within* a polygon. `contains` is the reverse (the polygon contains the point), used when the polygon is the left table.

3. Buffers and dissolve

`buffer(d)` grows each geometry by `d` metres; `dissolve(by=...)` merges rows that share an attribute.

```
buf = plots.copy()
buf["geometry"] = buf.buffer(3)           # 3-metre buffers
by_species = buf.dissolve(by="species")
print(by_species.geometry.area.round(1))  # merged area per species
```

4. Distance between geometries

```
d = plots.geometry.iloc[0].distance(plots.geometry.iloc[1])
print("distance p1 -> p2:", round(d, 2), "m")
```

🌲 **Why code this?** Spatial joins answer the questions foresters ask constantly: *which plots are inside the protected area? which streams cross this cutblock?* Buffers create riparian setbacks; dissolve merges stands into management units. This is the core of vector GIS — now in code you can re-run on next year's data.

☀ Challenge

Buffer the plots by **5 m**, then report the **total** buffered area (dissolve everything into one shape first so overlaps aren't double-counted).

▼ ☀ Show one solution

```
b = plots.copy()
b["geometry"] = plots.buffer(5)
one = b.dissolve()          # merge all into a single geometry
print("total buffered area:", round(one.geometry.area.iloc[0], 1), "m^2")
```

Nicely done

You've gone a step beyond the Chapter 4 material. Next chapter: making sure those areas and distances are trustworthy (CRS & data quality).

Chapter 5 — CRS, Files & Data Quality

This chapter makes your spatial data **trustworthy**: the right **coordinate reference system (CRS)** so measurements are correct, plus checks for invalid geometry, duplicates, and missing data. These skills are the foundation of **Lab 1**.

Run in **Python (gem530)**.

Learning outcomes

1. Explain geographic vs projected CRS and inspect/change a CRS.
2. Reproject and compute **correct** area/length.
3. Find and fix **invalid geometry**, **duplicates**, and **missing data**.
4. Use **relative paths** and a reproducible folder layout.
5. Run a small **QA checklist** function.

1. What CRS is my data in?

A **geographic** CRS (e.g. WGS84, `EPSG:4326`) uses degrees of lat/lon. A **projected** CRS (e.g. BC Albers `EPSG:3005`, or a UTM zone) uses metres — needed for real distances and areas.

```
import geopandas as gpd

# In practice: gdf = gpd.read_file("data/fires_points.gpkg") (your Chapter 4
# output).
# Here we build a small MKRF (British Columbia) sample so BC Albers (EPSG:3005)
# is correct:
gdf = gpd.GeoDataFrame(
    {"site": ["MKRF-1", "MKRF-2", "MKRF-3"]},
    geometry=gpd.points_from_xy([-122.57, -122.55, -122.52], [49.27, 49.29,
49.31]),
    crs="EPSG:4326",
)
```

```
print("CRS:", gdf.crs)
print("Is it geographic (degrees)?", gdf.crs.is_geographic)
```

🌲 **Why code this?** Knowing whether your data is in degrees or metres decides whether an 'area' means anything. Nearly every geospatial mistake starts with an unchecked CRS.

✅ Quick check — which CRS for area?

To compute area in square metres, do you use a geographic CRS (EPSG:4326) or a projected one (EPSG:3005)?

▼ ► Show answer

A **projected** CRS (e.g. EPSG:3005), whose units are metres. Geographic CRSs use degrees, which don't measure area.

2. Reproject and measure correctly

Degrees can't measure area. Reproject to a projected CRS, *then* measure. For polygons, compare the area before and after to see why this matters.

```
# BC Albers (metres) suits the MKRF area; Madagascar would use a UTM zone.
gdf_m = gdf.to_crs("EPSG:3005")
print("New CRS:", gdf_m.crs)
print("Now geographic?", gdf_m.crs.is_geographic)    # False -> metres

# Example with a polygon layer:
# stands = gpd.read_file("data/MKRF/stands.shp")
# print("Area in degrees^2 (wrong):", stands.area.iloc[0])
# print("Area in m^2 (correct):", stands.to_crs(3005).area.iloc[0])
```

Key idea. Always reproject to a projected CRS **before** computing area or distance. Reporting an area in "square degrees" is a classic mistake.

🌲 **Forestry example.** A 40-ha cutblock is 40 ha only in a *projected* CRS; measured in degrees the number is meaningless. Reproject, then measure — every area/volume estimate depends on it.

3. Data-quality checks

Real layers contain invalid geometries, duplicates, and gaps. Find them explicitly.

```
# invalid geometries?
invalid = gdf[~gdf.geometry.is_valid]
print("Invalid geometries:", len(invalid))

# a common repair for polygons: buffer(0) or make_valid()
# gdf["geometry"] = gdf.geometry.buffer(0)

# duplicate rows?
print("Duplicate rows:", gdf.duplicated().sum())

# missing attribute values per column
print("\nMissing values per column:")
print(gdf.isna().sum())
```

🌲 **Why code this?** Field datasets carry invalid polygons, duplicate plots, and missing values. Catching them in code protects every downstream total and map.

✅ Quick check — QA

What does `gdf.duplicated().sum()` tell you?

▼ ► Show answer

The **number of duplicate rows** — a quick check for repeated plots.

`gdf.isna().sum()` similarly counts missing values per column.

4. Reproducible paths & folders

Use **relative paths** (from your project folder), not absolute ones like `C:/Users/you/...`, so the project runs on any machine.

```
from pathlib import Path

project = Path.cwd()
data_dir = project / "data"
outputs = project / "outputs"
outputs.mkdir(exist_ok=True)

print("Project:", project.name)
print("Reading from:", data_dir.name, "| writing to:", outputs.name)
```

A tidy, reproducible layout:

```
project/
  data/      raw + cleaned inputs
  outputs/   results you generate
  notebook.ipynb
```

🌲 **Why code this?** Relative paths and a tidy data/outputs layout let your analysis re-run on a colleague's machine and next year's data — the definition of reproducible.

5. A reusable QA function

Wrap the checks so you can run them on any layer — you will reuse this in Lab 1.

```
def quick_qa(gdf, name="layer"):
    """Print a short data-quality report for a GeoDataFrame."""
    print(f"--- QA: {name} ---")
    print("rows:", len(gdf))
    print("CRS:", gdf.crs)
    print("invalid geometries:", (~gdf.geometry.is_valid).sum())
    print("duplicate rows:", gdf.duplicated().sum())
    print("missing values:", int(gdf.isna().sum().sum()))
```

```
quick_qa(gdf, "fires_points")
```

Try it yourself. Add one line to `quick_qa` that also prints whether the CRS is geographic or projected (hint: `gdf.crs.is_geographic`).

🌲 **Forestry example.** A reusable QA function is the checklist you run on every new survey layer before trusting it — consistency you can't get by eye.

🔧 Hands-on activity — QA a messy layer (~20 min)

The layer below has a duplicate plot and a missing measurement. Find and fix the problems, then reproject. Work individually.

Task 1. Report the number of duplicate rows and missing values. **Task 2.** Drop the duplicate rows into a new `clean` GeoDataFrame. **Task 3.** Reproject `clean` to EPSG:3005 and confirm it's no longer geographic.

```
import geopandas as gpd
gdf = gpd.GeoDataFrame(
    {"plot": [1, 2, 3, 3], "dbh_cm": [40.0, None, 55.0, 55.0]},
    geometry=gpd.points_from_xy([-122.57, -122.55, -122.52, -122.52],
                                [49.27, 49.29, 49.31, 49.31]),
    crs="EPSG:4326",
)
# TODO Task 1: report duplicates and missing values
# TODO Task 2: drop duplicate rows -> clean
# TODO Task 3: reproject clean to EPSG:3005; confirm not geographic
```

Activity solutions

▼ ► Task 1 — report

```
print("duplicates:", gdf.duplicated().sum())
print(gdf.isna().sum())
```

▼ ► Task 2 — drop duplicates

```
clean = gdf.drop_duplicates()
print(len(clean), "rows after dropping duplicates")
```

▼ ► Task 3 — reproject

```
clean_m = clean.to_crs("EPSG:3005")
print("geographic?", clean_m.crs.is_geographic) # False
```

Common errors this chapter (and the fix)

Message / symptom	Likely cause	Fix
area/distance is nonsense	measured in a geographic (degrees) CRS	<code>.to_crs("EPSG:3005")</code> then measure
spatial join returns nothing	layers in different CRSs	reproject one to match the other
works on my PC, not yours	absolute path like <code>C:/Users/you/...</code>	use a relative path from the project
<code>TopologyException</code>	invalid (self-intersecting) polygon	repair with <code>buffer(0)</code> / <code>make_valid</code>

Quick reference — Chapter 5 cheat sheet

```
gdf.crs # what CRS am I in?
gdf.crs.is_geographic # True = degrees, False = projected (metres)
gdf_m = gdf.to_crs("EPSG:3005") # reproject, THEN measure
gdf.geometry.is_valid # find invalid geometries
gdf.duplicated().sum() # count duplicate rows
gdf.isna().sum() # count missing values per column
```

```
from pathlib import Path
Path("outputs").mkdir(exist_ok=True) # relative, reproducible
```

Additional learning (optional)

Great if we have time, otherwise self-study. Not required for the quiz or exercise.

Bonus 1 — Real distances (reproject, then measure)

Why code this? "How far is each plot from the road?" only has a real answer in metres — i.e. in a projected CRS. This is the everyday reason we reproject.

```
import geopandas as gpd
pts = gpd.GeoSeries(
    gpd.points_from_xy([-122.57, -122.50], [49.27, 49.30]), crs="EPSG:4326"
).to_crs("EPSG:3005") # -> metres
d = pts.iloc[0].distance(pts.iloc[1])
print("distance between the two plots:", round(d), "m")
```

 In degrees this number would be meaningless; in a projected CRS it's real metres.

Bonus 2 — Repairing an invalid polygon

Why code this? Digitised stand boundaries often self-intersect ("bow-ties"), which breaks area and overlay tools. A one-line repair rescues them.

```
import geopandas as gpd
from shapely.geometry import Polygon
bowtie = Polygon([(0, 0), (2, 2), (2, 0), (0, 2)]) # self-intersecting
gdf = gpd.GeoDataFrame(geometry=[bowtie], crs="EPSG:3005")
print("valid before?", bool(gdf.is_valid.iloc[0]))
gdf["geometry"] = gdf.buffer(0) # common repair
print("valid after? ", bool(gdf.is_valid.iloc[0]))
```

🌲 Run this before any area calculation on messy boundary data.

Packages used this chapter

- **GeoPandas** — inspect/change CRS (`.to_crs`), validity (`.is_valid`), and run quality checks. *Geospatial link*: correctness of every measurement and join.
- **pyproj** — the CRS engine *behind* `.to_crs` . You rarely call it directly; know it is doing the reprojection. See the [Package Guide](#).

Working with AI (30-second habit)

AI often forgets the CRS. When AI gives you reprojection or area code, verify: (1) the source CRS is set, (2) you reproject **before** measuring, (3) the result units make sense (metres, not degrees).

Recap & what's next

Geographic vs projected CRS · reproject then measure · validity/duplicate/missing checks · relative paths & folders · a reusable QA function.

- **Practice**: Chapter 5 Exercise.
- **Next chapter (Chapter 6)**: consolidation + **Lab 1** studio — you have now learned everything Lab 1 assesses.

Practice exercises

Practise reprojection, correct measurement, and quality checks.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

Run in **Python (gem530)**.

Setup (run first)

```
import geopandas as gpd
# small MKRF (BC) sample so BC Albers (EPSG:3005) is correct:
gdf = gpd.GeoDataFrame({"site":["MKRF-1","MKRF-2","MKRF-3"]},
    geometry=gpd.points_from_xy([-122.57,-122.55,-122.52],[49.27,49.29,49.31]),
    crs="EPSG:4326")
```

1. Core

Reproject `gdf` to `EPSG:3005` (BC Albers, metres) and print the new CRS.

```
# TODO – your code here
```

2. Apply

Confirm the reprojected CRS is **not** geographic.

```
# TODO – your code here
```

3. Interpret

In a comment, explain why area/distance must be computed in a projected CRS, not degrees.

```
# TODO – your code here
```

4. Debug

A spatial join fails because the two layers are in different CRSs. Fix the pattern.

```
joined = gpd.sjoin(gdf, other)  # other is in a different CRS
```

```
# TODO – your code here
```

5. Extension

Write a function `quick_qa(gdf)` that prints row count, CRS, duplicate rows, and total missing values.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 5 exercise and want more.

You'll practise

1. See why **area must be measured in a projected CRS**.
2. Detect and repair **invalid geometries**.
3. Write and re-read a **GeoPackage** safely.

1. Area needs a projected CRS

Measuring area in degrees is meaningless. Reproject to metres first.

```
import geopandas as gpd
from shapely.geometry import Polygon
stand = gpd.GeoDataFrame({"id": [1]}, geometry=[Polygon(
    [(-122.6, 49.2), (-122.5, 49.2), (-122.5, 49.3), (-122.6, 49.3)]]),
    crs="EPSG:4326")
print("area in degrees^2 (meaningless):", round(stand.area.iloc[0], 6))

stand_m = stand.to_crs(3005)                # BC Albers, metres
print("area in ha (correct):", round(stand_m.area.iloc[0] / 10_000, 1))
```

✓ Quick check — CRS

You need the area of a harvest block in hectares. Do you compute it in EPSG:4326 (degrees) or EPSG:3005 (metres)?

▼ ✓ Show answer

EPSG:3005 (a projected, metre-based CRS). Area in degrees is not a real area — always `.to_crs()` to a projected CRS first.

2. Invalid geometries

A self-intersecting ("bow-tie") polygon breaks many operations. `buffer(0)` is a classic repair.

```
from shapely.geometry import Polygon
bowtie = Polygon([(0, 0), (2, 2), (2, 0), (0, 2)]) # self-intersecting
print("valid?", bowtie.is_valid)
fixed = bowtie.buffer(0)
print("fixed valid?", fixed.is_valid, "| area:", round(fixed.area, 2))
```

3. Round-trip to a GeoPackage

Write to disk and read back to confirm the CRS and rows survive. We use a temporary folder so nothing clutters your course folder.

```
import tempfile, os
folder = tempfile.mkdtemp()
path = os.path.join(folder, "stand.gpkg")
stand_m.to_file(path, driver="GPKG")
back = gpd.read_file(path)
print("re-read CRS:", back.crs.to_epsg(), "| rows:", len(back))
```

🌲 **Why code this?** A wrong CRS is the single most common geospatial bug — and it silently gives areas that are off by orders of magnitude. Checking validity and re-reading what you wrote is how you make a mangrove-loss or harvest-area number you can defend.

🌟 Challenge

Reproject `stand` to **UTM zone 10N (EPSG:32610)** and print its perimeter (boundary length) in metres.

▼ 🌟 Show one solution

```
u = stand.to_crs(32610)
print("perimeter:", round(u.geometry.length.iloc[0], 1), "m")
```

Nicely done

You've gone a step beyond the Chapter 5 material. You can now trust your measurements — next is consolidating everything into a workflow.

Chapter 6 — Consolidation

No new concepts this chapter. We connect Chapters 1–5 into one small, reproducible workflow, then you start **Lab 1**. This is a catch-up and confidence week before the ArcPy block.

Goals

- Chain the whole data-handling pipeline end to end.
- Diagnose the common errors (paths, CRS, merge keys, missing data).
- Begin Lab 1 in class with help available.

Class structure

Recap of the pipeline (20 min) · worked end-to-end demo (40) · **break** (10) · **Lab 1 studio** with TAs (70) · debugging clinic + AI-checking (20) · wrap (20).

The end-to-end pipeline (Chapters 1–5 in one flow)

Table → clean (Pandas) → points (GeoPandas) → reproject & QA (CRS) → summarize → export. Read each step and notice which chapter taught it.

```
import pandas as pd
import geopandas as gpd
from pathlib import Path

# 1) TABLE (Chapter 3): load and clean
# In practice: df = pd.read_csv("data/fires_clean.csv"). Small MKRF sample
# here:
df = pd.DataFrame({
    "site":  ["MKRF-N", "MKRF-N", "MKRF-S", "MKRF-S"],
    "lon":   [-122.57, -122.55, -122.52, -122.50],
    "lat":   [49.31, 49.32, 49.27, 49.28],
    "area_ha": [12.5, 3.0, 40.0, 22.0],
})
df = df.dropna(subset=["lat", "lon"])           # need coordinates to map
```

```
# 2) POINTS (Chapter 4): build a GeoDataFrame
gdf = gpd.GeoDataFrame(
    df, geometry=gpd.points_from_xy(df["lon"], df["lat"]), crs="EPSG:4326"
)

# 3) CRS + QA (Chapter 5): reproject, then check quality
gdf_m = gdf.to_crs("EPSG:3005")
print("invalid geometries:", (~gdf_m.geometry.is_valid).sum())
print("duplicate rows:", gdf_m.duplicated().sum())

# 4) SUMMARIZE (Chapter 3): area burned per site
print(gdf_m.groupby("site")["area_ha"].sum())

# 5) EXPORT (Chapters 4-5): reproducible output
Path("outputs").mkdir(exist_ok=True)
gdf_m.to_file("outputs/fires_bcalbers.gpkg", driver="GPKG")
print("Wrote outputs/fires_bcalbers.gpkg")
```

Debugging clinic — the usual suspects

When something breaks, check these first (each maps to a chapter):

1. **Path** wrong / absolute → *Chapter 5* (use relative paths).
2. **CRS** missing or mismatched → *Chapters 4–5* (`gdf.crs` , `.to_crs`).
3. **Merge key** wrong → *Chapter 3* (check the `on=` column and row counts).
4. **Missing values** → *Chapter 3* (`isna().sum()` , decide fill vs drop).
5. **(lon, lat) reversed** → *Chapters 1 & 4* (x = longitude first).

Lab 1 studio

Open **Lab 1** now and start Part 1 (tabular cleaning). Aim to finish Parts 1–2 in class while TAs are here. Lab 1 assesses only Chapters 1–5 — everything you need has been taught and practised.

No quiz this chapter (no new material). Use the time for Lab 1.

🌲 Why this consolidation chapter matters

Real forest analyses are never one command — they're a **chain**: table → clean → map → reproject → summarise → export. Most errors happen *between* the steps, not inside them. Today you practise the joins between steps, which is exactly what Lab 1 (and your project) will ask for.

🎓 Additional learning (optional)

Bonus — Wrap the whole pipeline in one reusable function

Why code this? Once a workflow works, package it. Next season's survey becomes a one-line call instead of a re-run of twenty cells — the difference between a script and a *tool*.

```
import pandas as pd, geopandas as gpd
from pathlib import Path

def plots_to_gpkg(df, out_path, to_crs="EPSG:3005"):
    """Table with lon/lat -> cleaned, reprojected GeoPackage. Returns the
    GeoDataFrame."""
    df = df.dropna(subset=["lon", "lat"])
    gdf = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df["lon"],
df["lat"]),
                        crs="EPSG:4326").drop_duplicates()
    gdf = gdf.to_crs(to_crs)
    Path(out_path).parent.mkdir(exist_ok=True, parents=True)
    gdf.to_file(out_path, driver="GPKG")
    return gdf

demo = pd.DataFrame({"site": ["A", "B"], "lon": [-122.57, -122.55], "lat":
[49.27, 49.29]})
result = plots_to_gpkg(demo, "outputs/demo_plots.gpkg")
print("rows:", len(result), "| CRS:", result.crs)
```

🌲 Reuse this on any new plot table — the same five steps, one call.

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 6 exercise and want more.

You'll practise

1. Chain **tabular** → **spatial** → **summary** in one clean pass.
2. Turn a CSV of coordinates into points, join them to regions, and summarise.
3. Practise the shape of a real analysis you'll repeat throughout.

1. Start with a table of plots

Imagine this came from a field CSV (`pd.read_csv`).

```
import pandas as pd, geopandas as gpd
from shapely.geometry import box
plots = pd.DataFrame({
    "plot": ["A", "B", "C", "D"],
    "lon": [-122.57, -122.55, 48.55, 48.58],
    "lat": [49.27, 49.29, -13.35, -13.33],
    "dbh_cm": [47, 61, 28, 40],
})
print(plots)
```

2. Make points, then region polygons

`points_from_xy` builds geometry straight from lon/lat columns.

```
gdf = gpd.GeoDataFrame(plots, geometry=gpd.points_from_xy(plots.lon,
plots.lat),
                        crs="EPSG:4326")
regions = gpd.GeoDataFrame({"region": ["MKRF", "Ambaro"]},
                           geometry=[box(-123, 49, -122, 50), box(48, -14, 49, -13)], crs="EPSG:4326")
print(gdf.geometry.geom_type.tolist())
```

3. Spatial join, then summarise by region

This is the payoff — a tabular summary that came from *where* the points are.

```
joined = gpd.sjoin(gdf, regions, predicate="within", how="left")
summary = joined.groupby("region")["dbh_cm"].agg(["count", "mean"]).round(1)
print(summary)
```

✓ Quick check — workflow

Why must both `gdf` and `regions` share the same CRS before `sjoin`?

▼ ✓ Show answer

A spatial join compares coordinates directly — if the two layers use different CRSs, the same real-world spot has different numbers and the join is wrong. Match with `.to_crs()` first.

🌲 **Why code this?** This tabular → spatial → summary pattern is Lab 1, and most real projects: read messy field data, put it on the map, and report a number per management unit. Master the shape here and every later analysis is a variation on it.

🔦 Challenge

Add a column `dbh_class` ("large" if DBH > 50 else "small"), then count large vs small plots **within each region**.

▼ 🔦 Show one solution

```
joined2 = joined.assign(dbh_class=joined["dbh_cm"].apply(lambda d: "large"
if d > 50 else "small"))
print(joined2.groupby(["region", "dbh_class"]).size())
```

🎓 Nicely done

You've gone a step beyond the Chapter 6 material. You've built a whole analysis end to end — exactly the muscle Lab 1 tests.

Chapter 7 — ArcPy: Tools & Environments

For two chapters we use **ArcPy** — ArcGIS Pro's Python package — to **automate** geoprocessing with scripts (no clicking through the GUI). Everything you did with GeoPandas/Rasterio has a place here too; ArcPy's distinct value is **scripting an existing GIS platform** and batch-automating repetitive tasks.

Environment: run this notebook from **ArcGIS Pro** → **Notebooks** (the `arcgispro-py3` Python), where the ArcGIS licence is active. It will **not** run in the `gem530` open-source environment. See `../../environments/README.md`.

The bundled `data/MKRF.gdb` holds the Malcolm Knapp Research Forest layers used here: `roads`, `streams`, `trails`, `hydroline` (all line features).

Learning outcomes

1. Import ArcPy and set the **workspace** and environment.
2. Call a **geoprocessing tool** from Python with parameters.
3. **List** datasets and fields in a workspace.
4. Use **Describe** to inspect a dataset.
5. Read **tool messages** and handle errors; name outputs systematically.

1. ArcPy is a package

You `import arcpy` like any package. The **workspace** is the default folder/geodatabase for inputs and outputs; `overwriteOutput` lets you re-run cells.

```
import arcpy

arcpy.env.workspace = r"data/MKRF.gdb"      # bundled MKRF geodatabase to the
provided geodatabase
```

```
arcpy.env.overwriteOutput = True
print("Workspace:", arcpy.env.workspace)
```

🌲 **Why code this?** Treating ArcPy as *just another package* is the mental shift: the same Python you've written for five chapters now drives a professional GIS.

2. Call a geoprocessing tool

Tools live in toolboxes (`analysis` , `management` , ...). Here we buffer roads by 100 m — the same idea as a GeoPandas buffer, but running ArcGIS's tool.

```
# arcpy.analysis.Buffer(in_features, out_feature_class, buffer_distance)
arcpy.analysis.Buffer("roads", "roads_buffer_100m", "100 Meters")
print("Created roads_buffer_100m")
print(arcpy.GetMessages())      # read what the tool reported
```

🌲 **Forestry example.** Buffering roads or streams, clipping stands to a watershed — the tools you'd click in ArcGIS become one repeatable line you can re-run on new data.

✓ Quick check — calling a tool

What's the modern way to call the Buffer tool?

▼ ► Show answer

```
arcpy.analysis.Buffer(in_fc, out_fc, "100 Meters")
```

 — tools live in toolbox modules (`analysis` , `management` , ...).

3. Inputs, outputs & systematic naming

Give outputs predictable names so batch results stay organized. Build names from the input + operation.

```
in_fc = "streams"
distance = "50 Meters"
out_fc = f"{in_fc}_buffer_{distance.split()[0]}m" # -> streams_buffer_50m
arcpy.analysis.Buffer(in_fc, out_fc, distance)
print("Created", out_fc)
```

🌲 **Why code this?** Systematic output names (`roads_buf_100m`) mean a batch of 50 outputs stays understandable six months later — a real problem in operational forestry.

4. List datasets and fields

Discover what's in a workspace — essential before automating over many layers.

```
feature_classes = arcpy.ListFeatureClasses()
print("Feature classes:", feature_classes)

for f in arcpy.ListFields("roads"):
    print(f.name, "|", f.type)
```

🌲 **Why code this?** Listing datasets and fields is how a script *discovers* what's in a geodatabase, so it can process whatever this year's delivery contains.

5. Describe a dataset

`Describe` returns properties (shape type, spatial reference, extent) — useful for decisions in a script.

```
desc = arcpy.Describe("roads")
print("Type:", desc.dataType)
print("Shape:", desc.shapeType)
print("CRS:", desc.spatialReference.name)
```

🌲 **Forestry example.** `Describe` tells your script the geometry type and CRS before it acts — so the same tool works on a stand polygon layer and a road line layer.

✅ Quick check — Describe

Name two things `arcpy.Describe("roads")` can tell you.

▼ ► Show answer

Its **shapeType** (Polyline/Polygon/Point) and its **spatialReference** (CRS) — plus data type and extent. Scripts branch on these.

6. Tool messages & error handling

Tools report messages; wrap risky calls in `try / except arcpy.ExecuteError` and print the messages to debug.

```
try:
    arcpy.analysis.Clip("streams", "study_area", "streams_clipped")
    print(arcpy.GetMessages())
except arcpy.ExecuteError:
    print("Tool error:\n", arcpy.GetMessages(2))    # 2 = error messages
```

🌲 **Why code this?** Tool messages and `try / except` are how an overnight batch over 200 layers reports what worked instead of dying silently.

🔧 Hands-on activity — Script a buffer workflow (~20 min)

Run inside **ArcGIS Pro** → **Notebooks** with the bundled `data/MKRF.gdb` (layers: `roads`, `streams`, `trails`, `hydroline`).

Task 1. Set the workspace and list the feature classes. **Task 2.** Buffer `streams` by 50 m with a systematic output name. **Task 3.** Describe your output and print its shape type and CRS.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True

# TODO Task 1: list the feature classes
# TODO Task 2: buffer streams by 50 m -> systematic name
# TODO Task 3: Describe the output
```

Activity solutions

▼ ► Task 1 — list

```
print(arcpy.ListFeatureClasses())
```

▼ ► Task 2 — buffer

```
in_fc, dist = "streams", "50 Meters"
out = f"{in_fc}_buf_{dist.split()[0]}m"
arcpy.analysis.Buffer(in_fc, out, dist)
print("created", out)
```

▼ ► Task 3 — describe

```
d = arcpy.Describe(out)
print(d.shapeType, "|", d.spatialReference.name)
```

Common errors this chapter (and the fix)

Symptom	Cause	Fix
"output already exists"	re-running a cell	<code>arcpy.env.overwriteOutput = True</code> , or a unique name
tool "not found"	wrong toolbox path	<code>arcpy.analysis.Buffer</code> , not <code>arcpy.Buffer</code>
<code>RuntimeError:</code> <code>dataset not found</code>	workspace/name wrong	check <code>arcpy.env.workspace</code> and <code>ListFeatureClasses()</code>
licence error	not running in ArcGIS Pro's Python	open ArcGIS Pro → Notebooks

Additional learning (optional)

Bonus 1 — Environment settings that save hours

Why code this? `arcpy.env` applies a setting to *every* later tool — clip everything to one study area, or snap all outputs to a common grid, without repeating parameters.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True
# arcpy.env.extent = "roads"          # process only this area
# arcpy.env.outputCoordinateSystem = arcpy.SpatialReference(3005)  # force BC Albers
print("workspace:", arcpy.env.workspace)
print("overwrite :", arcpy.env.overwriteOutput)
```

Bonus 2 — Check before you write

Why code this? In a long batch you want to skip work that's already done, and never crash on a missing input.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
for fc in ["roads", "streams", "does_not_exist"]:
    if arcpy.Exists(fc):
        print(fc, "-> found", ", ", arcpy.management.GetCount(fc)[0], "features")
```

```
else:
    print(fc, "-> MISSING, skipping")
```

🌲 `arcpy.Exists` is the guard that keeps an overnight batch alive.

Packages used this chapter

- **ArcPy** — ArcGIS Pro's Python package: run geoprocessing tools, inspect data (`Describe`, `ListFeatureClasses`, `ListFields`), and read tool messages. *Why*: automate a professional GIS platform from scripts. *When*: Chapters 7–8. *Geospatial link*: scripted geoprocessing over feature classes in a geodatabase. Requires an ArcGIS Pro licence. See the [Package Guide](#).

Working with AI (30-second habit)

AI often mixes up ArcPy versions/tool paths (e.g. `arcpy.Buffer_analysis` vs `arcpy.analysis.Buffer`). Confirm the tool name in the current **Esri documentation**, and read `arcpy.GetMessages()` rather than trusting that a tool "probably worked".

Recap & what's next

ArcPy as a package · workspace/environment · calling tools with parameters · listing datasets/fields · `Describe` · tool messages & errors · systematic naming.

- **Practice**: Chapter 7 Exercise (ArcPy — assessed by exercise, not a lab).
- **Next chapter**: automation — cursors and batch processing.

Practice exercises

Practise scripting geoprocessing tools with ArcPy.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

Run from **ArcGIS Pro** → **Notebooks** (`arcgispro-py3`), where the licence is active. Uses the bundled `data/MKRF.gdb` (layers: roads, streams, trails, hydroline).

Setup (run first)

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True
```

1. Core

Buffer the `roads` feature class by 100 metres to `roads_buf`, and print the tool messages.

```
# TODO – your code here
```

2. Apply

List all feature classes in the workspace, and list the fields of `roads`.

```
# TODO – your code here
```

3. Interpret

Use `Describe` on `roads` and print its shape type and CRS name (comment on what they mean).

```
# TODO – your code here
```

4. Debug

Re-running this errors because the output exists. Give **two** ways to fix it.

```
arcpy.analysis.Buffer("streams", "streams_buf", "50 Meters")
```

```
# TODO – your code here
```

5. Extension

Buffer `streams` with an output name built systematically from the input name and distance.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 7 exercise and want more.

You'll practise

1. Control tool behaviour with **arcpy.env** settings.
2. **Chain** geoprocessing tools (buffer then clip).
3. Inspect a dataset with **arcpy.Describe**.

⚠ **Run this in ArcGIS Pro → Notebooks** (Chapters 7–8 use `arcpy`, which only runs with an ArcGIS Pro licence). The code uses the bundled `data/MKRF.gdb` layers `roads`, `streams`, `hydroline`, `trails`.

1. Environment settings

`arcpy.env` values apply to every tool that follows — set them once at the top.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True          # let re-runs replace outputs
print("workspace:", arcpy.env.workspace)
print("feature classes:", arcpy.ListFeatureClasses())
```

2. Chain tools — buffer, then clip

The output of one tool becomes the input of the next. Here: buffer the roads, then clip the streams to that buffer.

```
road_buf = "road_buffer"
arcpy.analysis.Buffer("roads", road_buf, "50 Meters")
streams_near_roads = "streams_near_roads"
arcpy.analysis.Clip("streams", road_buf, streams_near_roads)
```

```
print("clipped streams count:",  
      int(arcpy.management.GetCount(streams_near_roads)[0]))
```

3. Describe a dataset

`Describe` reports a layer's type, geometry, and spatial reference — handy before you process it.

```
d = arcpy.Describe("roads")  
print("type:", d.dataType, "| shape:", d.shapeType)  
print("CRS:", d.spatialReference.name)
```

🌲 **Why code this?** Setting `arcpy.env` once (extent, snap raster, overwrite) keeps a multi-step forest workflow consistent and repeatable. Chaining `buffer` → `clip` is exactly how you'd extract *streams within 50 m of a road* for a riparian-risk assessment.

🌟 Challenge

Buffer `streams` by 30 m, then use **Clip** to keep only the `roads` that fall within that stream buffer. (Same pattern, inputs swapped.)

▼ 🌟 Show one solution

```
arcpy.analysis.Buffer("streams", "stream_buf", "30 Meters")  
arcpy.analysis.Clip("roads", "stream_buf", "roads_near_streams")  
print(int(arcpy.management.GetCount("roads_near_streams")[0]))
```

🎓 Nicely done

You've gone a step beyond the Chapter 7 material. Next: automating these tools over many datasets with cursors.

Chapter 8 — ArcPy: Automation

Last chapter we ran single tools. This chapter is ArcPy's real payoff: **automation** — read/edit attributes with **cursors**, run a tool over **many** datasets in a loop, and wrap workflows in **reusable functions**.

Environment: run this notebook from **ArcGIS Pro** → **Notebooks** (the `arcgispro-py3` Python), where the ArcGIS licence is active. It will **not** run in the `gem530` open-source environment. See `../../environments/README.md`.

The bundled `data/MKRF.gdb` holds the Malcolm Knapp Research Forest layers used here: `roads`, `streams`, `trails`, `hydroline` (all line features).

Learning outcomes

1. Read attributes with a **SearchCursor** and edit with an **UpdateCursor**.
2. **Loop** a tool over many datasets (batch processing).
3. Wrap a workflow in a **reusable function**.
4. Handle errors and log tool messages during automation.

1. Read attributes with a SearchCursor

A cursor iterates rows of a feature class. `SearchCursor` reads; you list the fields you want.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True

with arcpy.da.SearchCursor("roads", ["OBJECTID", "SHAPE@LENGTH", "NAME"]) as cur:
    for oid, length, name in cur:
        print(oid, round(length, 1), name)
```

🌲 **Why code this?** A cursor is how a script *reads the attribute table* — every road length, every stand's species — without opening it in ArcGIS.

✓ Quick check — cursors

Which cursor **reads** rows, and which one **changes** them?

▼ ► Show answer

`SearchCursor` reads; `UpdateCursor` reads *and* writes (call `cur.updateRow(row)`); `InsertCursor` adds new features.

2. Edit attributes with an UpdateCursor

`UpdateCursor` can change values row by row — e.g. fill a new length field.

```
# add a field, then populate it (roads are lines, so we use length)
arcpy.management.AddField("roads", "length_km", "DOUBLE")
with arcpy.da.UpdateCursor("roads", ["SHAPE@LENGTH", "length_km"]) as cur:
    for row in cur:
        row[1] = row[0] / 1000.0      # metres -> kilometres
        cur.updateRow(row)
print("Filled length_km")
```

🌲 **Forestry example.** Filling a `length_km` or `area_ha` field for thousands of features is a cursor loop, not a manual field-calculator session.

3. Batch processing: one tool over many datasets

The automation win: loop a tool over every feature class instead of clicking each.

```
for fc in arcpy.ListFeatureClasses():
    out = f"{fc}_buffer_100m"
    arcpy.analysis.Buffer(fc, out, "100 Meters")
    print("Buffered", fc, "->", out)
```

🌲 **Why code this?** Batch processing is the payoff of scripting: one loop buffers every layer in a geodatabase while you do something else.

✓ Quick check — batch safety

In a loop over many layers, why wrap each tool call in `try / except`?

▼ ► Show answer

So one bad layer is **logged and skipped** instead of killing the whole batch part-way through.

4. Wrap a workflow in a reusable function

Package a repeatable task so you can reuse it — the Chapter 2 idea, applied to geoprocessing.

```
def buffer_and_count(in_fc, distance="100 Meters"):
    """Buffer a feature class and return the output name + feature count."""
    out_fc = f"{in_fc}_buf"
    arcpy.analysis.Buffer(in_fc, out_fc, distance)
    n = int(arcpy.management.GetCount(out_fc)[0])
    return out_fc, n

name, count = buffer_and_count("roads", "75 Meters")
print(name, "has", count, "features")
```

🌲 **Why code this?** Wrapping a workflow in a function turns a one-off script into a tool you can reuse on next year's delivery.

5. Robust automation: errors & logging

When looping over many datasets, catch errors so one bad layer doesn't stop the whole batch, and log messages.

```
for fc in arcpy.ListFeatureClasses():
    try:
        arcpy.analysis.Buffer(fc, f"{fc}_buf", "100 Meters")
        print("OK:", fc)
    except arcpy.ExecuteError:
        print("FAILED:", fc, "->", arcpy.GetMessages(2))
```

Try it yourself. Change the batch buffer to write outputs into a separate "outputs" geodatabase, and add a print of each output's feature count.

🌲 **Forestry example.** Logging which layers succeeded and which failed is what makes an overnight batch trustworthy the next morning.

🔧 Hands-on activity — Automate the road network (~20 min)

Run in **ArcGIS Pro** → **Notebooks** with `data/MKRF.gdb`.

Task 1. Use a `SearchCursor` to print the 5 longest roads (`SHAPE@LENGTH`, `NAME`).

Task 2. Add a `length_km` field to `roads` and fill it with an `UpdateCursor`. **Task 3.** Buffer every feature class in the geodatabase by 100 m, skipping failures.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True

# TODO Task 1: SearchCursor -> longest roads
# TODO Task 2: AddField + UpdateCursor -> length_km
# TODO Task 3: batch buffer with try/except
```

Activity solutions

▼ ► Task 1 — longest roads

```
rows = []
with arcpy.da.SearchCursor("roads", ["SHAPE@LENGTH", "NAME"]) as cur:
    rows = sorted(cur, reverse=True)[:5]
for length, name in rows:
    print(round(length, 1), name)
```

▼ ► Task 2 — fill length_km

```
arcpy.management.AddField("roads", "length_km", "DOUBLE")
with arcpy.da.UpdateCursor("roads", ["SHAPE@LENGTH", "length_km"]) as cur:
    for row in cur:
        row[1] = row[0] / 1000.0
        cur.updateRow(row)
print("filled length_km")
```

▼ ► Task 3 — safe batch

```
for fc in arcpy.ListFeatureClasses():
    try:
        arcpy.analysis.Buffer(fc, fc + "_buf100", "100 Meters")
        print("OK:", fc)
    except arcpy.ExecuteError:
        print("FAILED:", fc, arcpy.GetMessages(2))
```

Common errors this chapter (and the fix)

Symptom	Cause	Fix
cursor hangs / "lock" error	a cursor left open	use <code>with</code> <code>arcpy.da.SearchCursor(...)</code> as <code>cur:</code>
<code>RuntimeError:</code> <code>Field not found</code>	field name typo	check with <code>arcpy.ListFields(fc)</code>
edits don't save	forgot <code>cur.updateRow(row)</code>	call it inside the loop

Symptom	Cause	Fix
batch dies on one layer	no error handling	wrap in <code>try/except</code> <code>arcpy.ExecuteError</code>

Additional learning (optional)

Bonus 1 — Adding new features with an `InsertCursor`

Why code this? Sometimes you must *create* data — a new sample-plot layer from GPS coordinates, or flag points from an analysis.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True
# make an empty point layer, then insert two plots
arcpy.management.CreateFeatureclass(arcpy.env.workspace, "new_plots", "POINT",

spatial_reference=arcpy.Describe("roads").spatialReference)
with arcpy.da.InsertCursor("new_plots", ["SHAPE@XY"]) as cur:
    cur.insertRow([(-122.57, 49.27)])
    cur.insertRow([(-122.55, 49.29)])
print("inserted:", arcpy.management.GetCount("new_plots")[0], "plots")
```

Bonus 2 — Progress logging for long batches

Why code this? A batch over hundreds of layers should tell you where it is and end with a summary you can trust.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
fcs = arcpy.ListFeatureClasses()
ok, failed = [], []
for i, fc in enumerate(fcs, start=1):
    print(f"[{i}/{len(fcs)}] {fc} ...")
    try:
        arcpy.analysis.Buffer(fc, fc + "_b", "25 Meters")
        ok.append(fc)
```

```
except arcpy.ExecuteError:
    failed.append(fc)
print("done. ok:", len(ok), "| failed:", failed)
```

🌲 This is what an operational overnight geoprocessing job looks like.

Packages used this chapter

- **ArcPy** (`arcpy.da` cursors, `management` / `analysis` tools) — read/edit attributes and automate geoprocessing across many datasets. *Geospatial link*: the batch/automation workflows that make scripting worthwhile vs clicking. See the [Package Guide](#).

Working with AI (30-second habit)

Cursor code is a common source of bugs: field names must match the data exactly, and an open cursor can **lock** a dataset. If AI cursor code hangs or errors, check field names against `ListFields`, and make sure cursors are inside a `with` block so they close.

Recap & what's next

SearchCursor (read) · UpdateCursor (edit) · batch loops · reusable functions · error handling & logging.

- **Practice:** Chapter 8 Exercise.
- **Next chapter:** raster data with NumPy & Rasterio (back in the `gem530` env).

Practice exercises

Practise cursors, batch processing, and reusable geoprocessing functions.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

Run from **ArcGIS Pro** → **Notebooks** (`arcgispro-py3`), where the licence is active. Uses the bundled `data/MKRF.gdb` (layers: roads, streams, trails, hydroline).

Setup (run first)

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True
```

1. Core

Use a `SearchCursor` to print the OBJECTID and length of each feature in `roads`.

```
# TODO – your code here
```

2. Apply

Add a `DOUBLE` field `length_km` to `roads` and fill it (length in m ÷ 1000) with an `UpdateCursor`.

```
# TODO – your code here
```

3. Interpret

Why must the field names passed to a cursor match the dataset exactly? Answer in a comment.

```
# TODO – your code here
```

4. Debug

This batch stops at the first bad layer. Make it skip failures and continue.

```
for fc in arcpy.ListFeatureClasses():  
    arcpy.analysis.Buffer(fc, fc + "_buf", "100 Meters")
```

```
# TODO – your code here
```

5. Extension

Wrap 'buffer + count features' in a reusable function `buffer_and_count(in_fc, distance)` returning the output name and count.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 8 exercise and want more.

You'll practise

1. Read attributes with a **SearchCursor**.
2. Edit attributes with an **UpdateCursor**.
3. **Batch**-process every feature class with a function.

⚠ **Run this in ArcGIS Pro → Notebooks.** Uses the bundled `data/MKRF.gdb` (`roads`, `streams`, `hydroline`, `trails`).

1. SearchCursor — total road length

A cursor streams through rows one at a time. `SHAPE@LENGTH` is the geometry's length.

```
import arcpy
arcpy.env.workspace = r"data/MKRF.gdb"
arcpy.env.overwriteOutput = True

total = 0.0
with arcpy.da.SearchCursor("roads", ["SHAPE@LENGTH"]) as cur:
    for (length,) in cur:
        total += length
print("total road length:", round(total, 1), "m")
```

2. UpdateCursor — add and fill a field

Add a `length_km` field, then write a value into every row.

```
if "length_km" not in [f.name for f in arcpy.ListFields("roads")]:
    arcpy.management.AddField("roads", "length_km", "DOUBLE")
```

```
with arcpy.da.UpdateCursor("roads", ["SHAPE@LENGTH", "length_km"]) as cur:
    for row in cur:
        row[1] = row[0] / 1000.0
        cur.updateRow(row)
print("length_km populated")
```

✓ Quick check — cursors

Which cursor would you use to *change* attribute values: `SearchCursor` or `UpdateCursor` ?

▼ ✓ Show answer

`UpdateCursor` — it can edit and write rows back with `updateRow`. `SearchCursor` is read-only.

3. Batch over every feature class

Wrap a tool in a loop to process a whole geodatabase at once.

```
def buffer_all(distance="25 Meters"):
    for fc in arcpy.ListFeatureClasses():
        out = fc + "_buf"
        arcpy.analysis.Buffer(fc, out, distance)
        print("buffered", fc, "->", out)

buffer_all()
```

🌲 **Why code this?** Cursors + batch loops are what make GIS *scale*: computing length for every road, or buffering every layer in a forest geodatabase, with no clicking. This is the automation payoff — one script re-runs the whole analysis when next year's data lands.

🌟 Challenge

Use a `SearchCursor` to find the **single longest** `streams` feature (its `SHAPE@LENGTH`).

▼ 🌟 **Show one solution**

```
longest = 0.0
with arcpy.da.SearchCursor("streams", ["SHAPE@LENGTH"]) as cur:
    for (length,) in cur:
        longest = max(longest, length)
print("longest stream:", round(longest, 1), "m")
```

🎓 **Nicely done**

You've gone a step beyond the Chapter 8 material. You can now automate ArcGIS tasks — back to open-source rasters next.

Chapter 9 — Raster Data with NumPy & Rasterio

Vector data was points and polygons. **Raster** data is a grid of cells — elevation, satellite bands, land cover. The key idea: **a raster band is a NumPy array**. **Rasterio** reads/writes the georeferenced files; **NumPy** does the math; **Matplotlib** displays it.

Run in **Python (gem530)**. The NumPy cells run anywhere; the Rasterio cells need a raster file (provided) and the gem530 environment.

Learning outcomes

1. Describe raster structure: bands, rows, columns, cells, NoData.
2. Read a raster band into a **NumPy array** and inspect metadata.
3. Do array math, **masking**, and simple **reclassification**.
4. Compute a simple index (e.g. a difference or NDVI).
5. **Write** a GeoTIFF and display it with Matplotlib.

1. A raster *is* an array

Let's build a tiny synthetic elevation raster (4×4) so the NumPy ideas are concrete. 255 marks NoData (cells with no valid value).

```
import numpy as np

band = np.array([
    [10, 12, 15, 255],
    [11, 13, 14, 255],
    [ 9, 10, 255, 255],
    [ 8,  9, 10, 11],
], dtype=float)
```

```
print("shape (rows, cols):", band.shape)
print("min/max (incl. NoData):", band.min(), band.max())
```

🌲 **Why code this?** Elevation, canopy height, burn severity, satellite bands — all arrive as grids. Seeing a raster as an **array** is what lets you compute on millions of forest pixels at once.

2. Handle NoData with a mask

Replace the NoData value with `np.nan` so it doesn't corrupt statistics.

```
nodata = 255
masked = np.where(band == nodata, np.nan, band)
print("mean elevation (NoData ignored):", round(np.nanmean(masked), 2))
```

🌲 **Forestry example.** Cloud, shadow and off-image pixels are NoData. Masking them is why your mean canopy height describes the *forest*, not the holes in the data.

✓ Quick check — NoData

Why replace a NoData value (255) with `np.nan` before averaging?

▼ ► Show answer

NoData isn't a measurement — leaving 255 in would drag the mean upward. Mask it, then use `np.nanmean()` so only real pixels count.

3. Array math & reclassification

Operations apply to every cell at once. Here we reclassify into low/high zones.

```
# add 100 m to every valid cell (array math)
raised = masked + 100
print(raised)
```

```
# reclassify: 1 = low (<12), 2 = high (>=12); NoData stays NaN
reclass = np.where(np.isnan(masked), np.nan, np.where(masked < 12, 1, 2))
print("\nReclassified:\n", reclass)
```

🌲 **Why code this?** Reclassifying continuous values into classes — low/moderate/high burn severity, slope classes — is how raw measurements become **management categories**.

4. A simple index (NDVI)

Vegetation indices combine bands. $NDVI = (NIR - Red) / (NIR + Red)$. With two small synthetic bands:

```
red = np.array([[0.20, 0.22], [0.19, 0.25]])
nir = np.array([[0.40, 0.55], [0.38, 0.60]])
ndvi = (nir - red) / (nir + red)
print("NDVI:\n", np.round(ndvi, 3))
```

🌲 **Forestry example.** NDVI turns raw bands into a vegetation-vigour map: the workhorse index for monitoring forest health, drought stress and regrowth after harvest or fire.

5. Read a real raster with Rasterio

The real workflow: open a GeoTIFF, read a band into an array, and inspect its georeferencing (CRS, transform, NoData).

```
import rasterio
from rasterio.transform import from_origin
from pathlib import Path
Path("data").mkdir(exist_ok=True)

# In practice you would open a provided raster (e.g. SRTM over MKRF). Here we
# create a small demo GeoTIFF so the notebook runs on its own:
```

```

demo = np.random.default_rng(0).normal(300, 50, (30, 30)).astype("float32")
demo[0:3, 0:3] = -9999.0
with rasterio.open("data/demo_elev.tif", "w", driver="GTiff", height=30,
width=30,
                    count=1, dtype="float32", crs="EPSG:4326",
                    transform=from_origin(-122.75, 49.40, 0.01, 0.01),
nodata=-9999.0) as dst:
    dst.write(demo, 1)

with rasterio.open("data/demo_elev.tif") as src:
    elev = src.read(1)          # band 1 -> a NumPy array
    print("shape:", elev.shape, "| CRS:", src.crs, "| NoData:", src.nodata)
    profile = src.profile       # keep georeferencing for writing later

```

🌲 **Why code this?** Rasterio keeps pixels tied to real-world coordinates. Without the CRS and transform your result is a picture; with them it's a **map** you can overlay on stands and roads.

✅ Quick check — Rasterio

What does `src.read(1)` give you?

▼ ► Show answer

Band 1 as a NumPy array. The dataset object also carries `src.crs`, `src.transform` and `src.nodata`.

6. Write a result back to a GeoTIFF

Compute something, then save it **with the same georeferencing** so it lines up in any GIS.

```

# reclassify the raster we read (elev) and write it out with the SAME profile
elev_masked = np.where(elev == -9999.0, np.nan, elev)
elev_reclass = np.where(np.isnan(elev_masked), -9999.0,
                        np.where(elev_masked < 300, 1, 2)).astype("float32")

Path("outputs").mkdir(exist_ok=True)

```

```

profile.update(dtype="float32")          # reuse CRS + transform from the
source
with rasterio.open("outputs/elev_reclass.tif", "w", **profile) as dst:
    dst.write(elev_reclass, 1)

with rasterio.open("outputs/elev_reclass.tif") as chk:  # re-open to confirm
    print("wrote + re-opened:", chk.read(1).shape, "| CRS kept:", chk.crs)

```

🌲 **Why code this?** Writing results back to GeoTIFF (same georeferencing) is how your analysis becomes a layer a colleague opens in QGIS or ArcGIS.

7. Display with Matplotlib

A quick look at a raster array (full mapping is Chapter 10).

```

import matplotlib.pyplot as plt

plt.figure(figsize=(4, 4))
plt.imshow(masked, cmap="terrain")
plt.colorbar(label="elevation (m)")
plt.title("Synthetic elevation")
plt.show()

```

Try it yourself. Change the reclassification threshold from 12 to 11 and re-run. How many cells change class?

🌲 **Forestry example.** A quick colour-mapped display is the fastest way to sanity-check a burn-severity or NDVI result *before* you trust a number from it.

🔧 Hands-on activity — Burn-severity classes (~20 min)

A small burn-severity grid is below (`-1` marks NoData). Work individually.

Task 1. Mask NoData to `np.nan` and report the mean severity. **Task 2.** Reclassify into 1 = low (<0.3), 2 = moderate ($0.3-0.6$), 3 = high (>0.6). **Task 3.** Count the pixels in each class.

```
import numpy as np
severity = np.array([
    [0.10, 0.45, 0.80, -1.0],
    [0.25, 0.55, 0.72, 0.90],
    [-1.0, 0.35, 0.20, 0.65],
    [0.05, 0.15, 0.50, 0.75],
])
# TODO Task 1: mask -1 -> np.nan, print the mean
# TODO Task 2: reclassify into 1 / 2 / 3
# TODO Task 3: count pixels per class
```

Activity solutions

▼ ► Task 1 — mask & mean

```
masked = np.where(severity == -1.0, np.nan, severity)
print("mean severity:", round(np.nanmean(masked), 3))
```

▼ ► Task 2 — reclassify

```
cls = np.where(np.isnan(masked), np.nan,
               np.where(masked < 0.3, 1, np.where(masked <= 0.6, 2, 3)))
print(cls)
```

▼ ► Task 3 — counts

```
for k in (1, 2, 3):
    print("class", k, "->", int(np.nansum(cls == k)), "pixels")
```

Common errors this chapter (and the fix)

Symptom	Cause	Fix
statistics look wrong	NoData included	mask to <code>np.nan</code> , use <code>np.nanmean</code>
<code>ValueError: operands could not be broadcast</code>	arrays different shapes	check <code>arr.shape</code> before combining
output won't line up in GIS	wrote without the profile	reuse <code>src.profile</code> when writing
all-black / all-white image	one extreme value stretches the colours	mask NoData before plotting

Additional learning (optional)

Bonus 1 — From pixel counts to hectares

Why code this? Managers ask "how many hectares burned?", not "how many pixels?". With a known pixel size the conversion is one line — and it's how burn/harvest areas get reported.

```
import numpy as np
cls = np.array([[1, 2, 3], [2, 3, 3], [1, 1, 2]], dtype=float)
pixel_m = 30                                # Landsat pixels are 30 x 30 m
area_ha_per_pixel = (pixel_m * pixel_m) / 10_000    # m2 -> hectares
for k in (1, 2, 3):
    n = int(np.nansum(cls == k))
    print(f"class {k}: {n} pixels = {n * area_ha_per_pixel:.2f} ha")
```

Bonus 2 — Zonal statistics (a value inside a zone)

Why code this? "What's the mean NDVI *inside this stand*?" is the classic zonal question — a boolean mask plus `np.nanmean` answers it.

```
import numpy as np
ndvi = np.array([[0.2, 0.6, 0.7], [0.3, 0.8, 0.9], [0.1, 0.4, 0.5]])
stand = np.array([[False, True, True], [False, True, True], [False, False, False]])
```

```
inside = np.where(stand, ndvi, np.nan)
print("mean NDVI inside the stand:", round(np.nanmean(inside), 3))
print("mean NDVI overall      :", round(np.nanmean(ndvi), 3))
```

🌲 Swap the boolean mask for a rasterised stand boundary and this is production zonal stats.

Packages used this chapter

- **NumPy** — n-dimensional arrays and fast cell-wise math; a raster band is a NumPy array. *Geospatial link*: all raster analysis (indices, masking, reclassify).
- **Rasterio** — read/write georeferenced rasters (GeoTIFF) as arrays, preserving CRS + transform + NoData. *Geospatial link*: the bridge between raster files and NumPy.
- **Matplotlib** — display arrays (`imshow`). See the [Package Guide](#).

Working with AI (30-second habit)

Raster bugs cluster around **NoData**, **dtype**, and **array shape**. If AI code gives odd statistics, check whether NoData was masked and whether the arrays have matching shapes before you combine them.

Recap & what's next

Raster = array · NoData masking · array math & reclassify · a simple index · read/ write GeoTIFF with Rasterio · display with Matplotlib.

- **Practice**: Chapter 9 Exercise.
- **Next chapter**: turn all of this into clear maps — visualization & communication.

Practice exercises

Practise raster-as-array operations: NoData, reclassify, indices.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

NumPy parts run anywhere; the Rasterio part needs the `gem530` env + a raster file.

Setup (run first)

```
import numpy as np
band = np.array([[5, 7, 9, 255],[6, 8, 255, 255],[4, 5, 6, 7],[3, 4, 5, 255]],
dtype=float)
nodata = 255
```

1. Core

Mask the NoData value to `np.nan` and print the mean of the valid cells.

```
# TODO – your code here
```

2. Apply

Reclassify `masked` into 1 (`< 6`) and 2 (`>= 6`), keeping NaN as NaN.

```
# TODO – your code here
```

3. Interpret

How many valid cells are class 2? Print the count (comment on it).

```
# TODO – your code here
```

4. Debug

This mean is wrong because NoData wasn't handled. Fix it.

```
print(band.mean())
```

```
# TODO – your code here
```

5. Extension

Given red/nir arrays, compute $NDVI = (nir - red) / (nir + red)$.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 9 exercise and want more.

You'll practise

1. Compute **NDVI** from red and NIR bands.
2. Handle **NoData** with `np.where` and `np.nanmean`.
3. **Reclassify** by thresholds and read a terrain **gradient**.
4. Write and re-read a raster fully **in memory** (no files).

1. NDVI — vegetation from two bands

$NDVI = (NIR - Red) / (NIR + Red)$. NumPy applies it to whole arrays at once.

```
import numpy as np
red = np.array([[0.20, 0.30], [0.25, 0.28]])
nir = np.array([[0.50, 0.45], [0.60, 0.40]])
ndvi = (nir - red) / (nir + red)
print(np.round(ndvi, 3))
```

2. Masking NoData

Sentinel values like `-9999` must be excluded, or they wreck your statistics.

```
elev = np.array([[110, 112, -9999], [108, -9999, 111]], dtype=float)
masked = np.where(elev == -9999, np.nan, elev)
print("mean ignoring NoData:", round(np.nanmean(masked), 1))
```

✓ Quick check — NoData

Why use `np.nanmean` instead of `np.mean` after masking NoData to `np.nan`?

▼ ✓ Show answer

`np.mean` returns `nan` if any value is `nan`; `np.nanmean` **skips** the NaNs and averages the real pixels.

3. Reclassify + terrain gradient

Threshold the elevation into classes, and use `np.gradient` for a simple slope-direction read.

```
recl = np.where(np.isnan(masked), np.nan, np.where(masked < 110, 1, 2))
print("reclassified:\n", recl)

dem = np.array([[1., 2, 3], [2, 3, 4], [3, 4, 5]])
dz_dy, dz_dx = np.gradient(dem)
print("gradient in x:\n", dz_dx)
```

4. Write a raster in memory

`MemoryFile` lets you create and read a GeoTIFF without touching disk — great for tests and pipelines.

```
import rasterio
from rasterio.io import MemoryFile
from rasterio.transform import from_origin
arr = np.nan_to_num(recl, nan=0).astype("float32")
with MemoryFile() as mem:
    with mem.open(driver="GTiff", height=arr.shape[0], width=arr.shape[1],
count=1,
dtype="float32", crs="EPSG:3005",
transform=from_origin(0, 10, 10, 10)) as ds:
        ds.write(arr, 1)
    with mem.open() as ds:
        print("read back:", ds.read(1).shape, "| CRS:", ds.crs.to_epsg())
```

🌲 **Why code this?** Band math turns raw satellite bands into meaning — NDVI for greenness, NBR for burn severity. Masking NoData keeps cloud and edge pixels out

of your averages. These array moves are the whole of raster analysis, whether the grid is elevation, canopy height, or a Landsat scene.

🌟 Challenge

Compute NDVI for the arrays below, then report the **fraction of pixels with NDVI > 0.4** (healthy vegetation).

▼ 🌟 Show one solution

```
red = np.array([[0.2, 0.4], [0.1, 0.35]])
nir = np.array([[0.6, 0.5], [0.7, 0.4]])
ndvi = (nir - red) / (nir + red)
frac = (ndvi > 0.4).mean()
print("NDVI:\n", np.round(ndvi, 2))
print("fraction > 0.4:", round(float(frac), 2))
```

🎓 Nicely done

You've gone a step beyond the Chapter 9 material. Next chapter: turning these grids and vectors into clear, publishable maps.

Chapter 10 — Visualization & Communication

A result only matters if you can **communicate** it. This chapter: clear maps of vector and raster data, thoughtful classification and legends, and a short honest interpretation. **Matplotlib** is our main tool; `GeoDataFrame.explore()` gives one quick interactive map.

Run in **Python (gem530)**.

Learning outcomes

1. Plot vector data and make a classified **choropleth** with a legend.
2. Display raster data and **overlay** vector on it.
3. Add titles, labels, and an annotation.
4. Plot an attribute summary (a chart from a table).
5. Make one interactive map with `.explore()` and write a 3-sentence interpretation.

1. Plot vector data

`GeoDataFrame.plot()` draws geometries. Colour by a column to make a **choropleth**; `scheme="quantiles"` (via `mapclassify`) groups values into classes.

```
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt

# In practice: gdf = gpd.read_file("outputs/fires_bcalbers.gpkg") (Chapter 6
output).
# Small MKRF sample so the classification has enough values to show:
lons = np.linspace(-122.60, -122.48, 8)
lats = np.linspace(49.26, 49.34, 8)
```

```

gdf = gpd.GeoDataFrame(
    {"site": ["MKRF"] * 8, "area_ha": [5, 8, 12, 3, 20, 15, 7, 25]},
    geometry=gpd.points_from_xy(lons, lats), crs="EPSG:4326")

fig, ax = plt.subplots(figsize=(6, 6))
gdf.plot(ax=ax, column="area_ha", scheme="quantiles", k=4,
         legend=True, markersize=40, cmap="OrRd")
ax.set_title("Fire size by location (quantile classes)")
ax.set_axis_off()
plt.show()

```

🌲 **Why code this?** A map is how forestry results get *used*. Plotting straight from your GeoDataFrame means the figure always matches the data — no re-exporting to another program.

✅ Quick check — choropleth

What does `column="area_ha"` do in `gdf.plot(column="area_ha", legend=True)`?

▼ ► Show answer

It colours each feature by its `area_ha` value (a **choropleth**) and draws a matching legend.

2. Display raster + overlay vector

Plot a raster with `rasterio.plot.show`, then draw vector features on the **same axes** so they line up (they must share a CRS).

```

import rasterio
from rasterio.plot import show
from rasterio.transform import from_origin
from pathlib import Path
Path("data").mkdir(exist_ok=True)

# In practice: open a provided DEM (e.g. SRTM.tif). Here we make a small demo
# raster covering the sample points so the overlay lines up:

```

```

demo = np.random.default_rng(1).normal(300, 40, (20, 20)).astype("float32")
with rasterio.open("data/demo_elev.tif", "w", driver="GTiff", height=20,
width=20,
                    count=1, dtype="float32", crs="EPSG:4326",
                    transform=from_origin(-122.62, 49.36, 0.008, 0.006)) as dst:
    dst.write(demo, 1)

fig, ax = plt.subplots(figsize=(6, 6))
with rasterio.open("data/demo_elev.tif") as src:
    show(src, ax=ax, cmap="terrain") # raster
backdrop
    gdf.to_crs(src.crs).plot(ax=ax, color="red", markersize=30) # overlay
ax.set_title("Fire points over elevation")
plt.show()

```

🌲 **Forestry example.** Stands over a hillshade, fire points over burn severity — overlaying vector on raster is how you show *where* something happened, in context.

✅ Quick check — overlays

Two layers plot in the same figure but don't line up. First thing to check?

▼ ► Show answer

Their **CRS**. Reproject one to match the other (`gdf.to_crs(other.crs)`) and draw both on the same `ax` .

3. Titles, labels, annotation

Every analytical map needs a **title**, a **legend**, and often one **annotation** pointing at the key feature.

```

fig, ax = plt.subplots(figsize=(6, 6))
gdf.plot(ax=ax, column="area_ha", legend=True, markersize=40, cmap="OrRd")
ax.set_title("Burned area by fire (ha)")
# annotate the largest fire
biggest = gdf.loc[gdf["area_ha"].idxmax()]
ax.annotate("largest fire",

```

```

xy=(biggest.geometry.x, biggest.geometry.y),
xytext=(10, 10), textcoords="offset points",
arrowprops=dict(arrowstyle="->"))
plt.show()

```

🌲 **Why code this?** A title, legend and one annotation are the difference between a plot only you understand and a figure that stands alone in a report.

4. Plot an attribute summary

Sometimes a chart communicates better than a map. Pandas plots straight from a summary table.

```

by_site = gdf.groupby("site")["area_ha"].sum()
by_site.plot(kind="bar", title="Total burned area per site (ha)")
plt.ylabel("area (ha)")
plt.tight_layout()
plt.show()

```

🌲 **Forestry example.** Sometimes a bar chart of area-by-species communicates better than a map. Choose the form that answers the question.

5. One interactive map

`GeoDataFrame.explore()` builds an interactive (Folium/Leaflet) map with one line — you don't need to learn Folium's API. Good for exploring; static maps are still best for reports.

```

m = gdf.to_crs(4326).explore(column="area_ha", cmap="OrRd", legend=True)
m # displays an interactive, zoomable map in the notebook

```

🌲 **Why code this?** An interactive map lets a colleague zoom to their own

management unit — perfect for exploration and review meetings.

6. Communicate the result

A map needs words. Write **3 sentences**: what the map shows, one pattern you see, and one limitation (e.g. small sample, missing data, class choices affect the look).

Try it yourself. Change the choropleth to `k=3` classes and a different `cmap`. Does the story the map tells change? That's why classification choices must be reported.

🌲 **Why code this?** Stating the pattern *and* a limitation is what makes a map honest. Class breaks and colour choices can exaggerate — say what you chose.

🔧 Hands-on activity — Build an analytical map (~20 min)

Make a map that could go in a report. Work individually.

Task 1. Plot the stands below coloured by `area_ha`, with a legend. **Task 2.** Add a title and turn the axes off. **Task 3.** Annotate the largest stand.

```
import geopandas as gpd, matplotlib.pyplot as plt
from shapely.geometry import box
stands = gpd.GeoDataFrame(
    {"stand": ["A", "B", "C"], "area_ha": [12.0, 45.0, 28.0]},
    geometry=[box(0, 0, 1, 1), box(1.2, 0, 2.6, 1.4), box(0, 1.3, 1.1, 2.3)],
    crs="EPSG:3005",
)
# TODO Task 1: choropleth by area_ha with a legend
# TODO Task 2: title + axes off
# TODO Task 3: annotate the largest stand
```

Activity solutions

▼ ► Task 1 — choropleth

```
ax = stands.plot(column="area_ha", legend=True, cmap="YlGn",  
edgecolor="black")
```

▼ ► Task 2 — title & axes

```
ax.set_title("Stand area (ha)")  
ax.set_axis_off()
```

▼ ► Task 3 — annotate

```
big = stands.loc[stands["area_ha"].idxmax()]  
c = big.geometry.centroid  
ax.annotate("largest stand", xy=(c.x, c.y), xytext=(6, 6),  
            textcoords="offset points", arrowprops=dict(arrowstyle="->"))  
plt.show()
```

Common errors this chapter (and the fix)

Symptom	Cause	Fix
layers don't overlap	different CRSs	<code>gdf.to_crs(other.crs)</code> and share one <code>ax</code>
no legend appears	forgot <code>legend=True</code>	add it (and <code>column=</code>)
map is unreadable	too many classes / harsh colours	fewer classes, a sequential colormap
figure cut off when saved	tight elements	<code>plt.savefig(..., bbox_inches="tight")</code>

Additional learning (optional)

Bonus 1 — Save a report-quality figure

Why code this? A screenshot won't survive a report. `savefig` with a sensible DPI gives you a crisp, embeddable figure.

```
import geopandas as gpd, matplotlib.pyplot as plt
from shapely.geometry import box
from pathlib import Path
g = gpd.GeoDataFrame({"v": [1, 2]}, geometry=[box(0,0,1,1), box(1.2,0,2,1)],
crs="EPSG:3005")
ax = g.plot(column="v", legend=True, edgecolor="black")
ax.set_title("Example stand map"); ax.set_axis_off()
Path("outputs").mkdir(exist_ok=True)
plt.savefig("outputs/stand_map.png", dpi=300, bbox_inches="tight")
plt.close()
print("wrote outputs/stand_map.png (300 dpi)")
```

Bonus 2 — Before / after side by side

Why code this? Change is the story in forestry — harvest, fire, regrowth. Two panels sharing a colour scale let a reader compare dates honestly.

```
import numpy as np, matplotlib.pyplot as plt
before = np.array([[0.8, 0.7], [0.75, 0.85]])
after = np.array([[0.3, 0.65], [0.20, 0.80]])
fig, axes = plt.subplots(1, 2, figsize=(7, 3))
for ax, arr, name in zip(axes, [before, after], ["Before", "After"]):
    im = ax.imshow(arr, vmin=0, vmax=1, cmap="YlGn") # same scale on both!
    ax.set_title(name); ax.set_axis_off()
fig.colorbar(im, ax=axes, label="NDVI", shrink=0.8)
plt.show()
```

🌲 Note `vmin` / `vmax` — using the *same* scale on both panels is what makes the comparison fair.

Packages used this chapter

- **Matplotlib** — the primary plotting tool: maps, overlays, legends, annotations, charts. *Geospatial link*: renders both vector (via GeoPandas) and raster (via Rasterio) results.
- **GeoPandas** `.plot` / `.explore` — choropleths and one-line interactive maps (`.explore` uses Folium under the hood, so we don't teach Folium separately).
- **mapclassify** — classification schemes (`quantiles`, `equal_interval`) for choropleths. See the [Package Guide](#).

Working with AI (30-second habit)

AI makes plausible plotting code, but check: does the **legend** match the data, is the **CRS** consistent for overlays, and are the **class breaks** honest (not chosen to exaggerate)? Misleading maps are easy to make by accident.

Recap & what's next

Vector plots & choropleths · raster display + overlay · titles/legends/annotation · attribute charts · one `.explore()` map · honest interpretation.

- **Practice:** Chapter 10 Exercise. **Lab 2** is released this chapter.
- **Next chapter:** an introduction to machine learning with environmental data.

Practice exercises

Practise clear maps, overlays, and interpretation.

Structure: 1) core · 2) apply · 3) interpret · 4) debug · 5) extension. ~30–60 min, individual.

Run in **Python (gem530)**. Uses `outputs/points.gpkg` from the Chapter 4 exercise (or any point layer).

Setup (run first)

```
import numpy as np, geopandas as gpd, matplotlib.pyplot as plt
lons=np.linspace(-122.60,-122.48,8); lats=np.linspace(49.26,49.34,8)
gdf = gpd.GeoDataFrame({"region":["MKRF"]*5+["Ambaro"]*3, "area_ha":
[5,8,12,3,20,15,7,25]}),
    geometry=gpd.points_from_xy(lons,lats), crs="EPSG:4326")
```

1. Core

Plot `gdf` coloured by `region` with a legend and a title.

```
# TODO – your code here
```

2. Apply

Make a bar chart of the number of points per region.

```
# TODO – your code here
```

3. Interpret

In a Markdown answer (comment here), state one pattern the map shows and one limitation.

```
# TODO – your code here
```

4. Debug

This overlay misaligns because CRSs differ. Fix the pattern.

```
gdf.plot(ax=ax); raster_layer_in_other_crs.plot(ax=ax)
```

```
# TODO – your code here
```

5. Extension

Make one interactive map with `.explore()` coloured by `region`.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 10 exercise and want more.

You'll practise

1. Build **multi-panel** figures with subplots.
2. Make a **classified** choropleth with a mapclassify scheme.
3. **Overlay** vector points on a raster.

1. A two-panel figure

We use the non-interactive `Agg` backend so this runs anywhere (no pop-up windows).

```
import numpy as np, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import geopandas as gpd
from shapely.geometry import Point

rng = np.random.default_rng(0)
gdf = gpd.GeoDataFrame({"v": [1, 2, 3, 4, 5]},
                       geometry=[Point(x, x) for x in range(5)], crs="EPSG:3005")

fig, ax = plt.subplots(1, 2, figsize=(8, 3))
ax[0].imshow(rng.random((10, 10)), cmap="viridis"); ax[0].set_title("a raster")
gdf.plot(ax=ax[1], column="v", cmap="YlGnBu", legend=True);
ax[1].set_title("points")
plt.tight_layout()
print("built a 2-panel figure:", len(fig.axes), "axes")
```

2. A classified choropleth

A **classification scheme** (e.g. quantiles) groups values into bins — far clearer than a raw continuous ramp.

```

polys = gpd.GeoDataFrame({"area_ha": [5, 20, 35, 50, 80]},
    geometry=gpd.GeoSeries.from_wkt([
        "POLYGON((0 0,1 0,1 1,0 1,0 0))", "POLYGON((1 0,2 0,2 1,1 1,1 0))",
        "POLYGON((2 0,3 0,3 1,2 1,2 0))", "POLYGON((3 0,4 0,4 1,3 1,3 0))",
        "POLYGON((4 0,5 0,5 1,4 1,4 0))"]), crs="EPSG:3005")
ax = polys.plot(column="area_ha", scheme="quantiles", k=3, cmap="Reds",
    legend=True)
print("classified choropleth drawn with", "quantiles (k=3)")

```

✓ Quick check — classification

Why classify a choropleth into bins instead of a continuous colour ramp?

▼ ✓ Show answer

Bins make categories readable and comparable across maps; a continuous ramp hides where the meaningful breaks are. Schemes like `quantiles` or `natural_breaks` choose those breaks for you.

3. Overlay points on a raster

Draw the raster first, then plot the vector *on the same axes* (`ax=ax`).

```

import rasterio
from rasterio.io import MemoryFile
from rasterio.transform import from_origin
from rasterio.plot import show

arr = rng.random((20, 20)).astype("float32")
with MemoryFile() as mem:
    with mem.open(driver="GTiff", height=20, width=20, count=1,
        dtype="float32",
            crs="EPSG:3005", transform=from_origin(0, 20, 1, 1)) as ds:
        ds.write(arr, 1)
    with mem.open() as ds:
        # reopen read-only for plotting
        fig, ax = plt.subplots()
        show(ds, ax=ax, cmap="terrain")
        gdf.plot(ax=ax, color="red", markersize=30)
print("overlaid", len(gdf), "points on the raster")

```

🌲 **Why code this?** A map is how your analysis reaches a decision-maker. Multi-panel layouts compare before/after or two indices side by side; classified choropleths make *area burned per district* legible at a glance; overlays put your sample plots in context on the terrain. Communication is half the job.

🌟 Challenge

Re-draw the choropleth from section 2 using the **natural_breaks** scheme with **k=4**, and a `viridis` colour map.

▼ 🌟 Show one solution

```
ax = polys.plot(column="area_ha", scheme="natural_breaks", k=4,
                 cmap="viridis", legend=True)
print("drew natural_breaks choropleth, k=4")
```

🎓 Nicely done

You've gone a step beyond the Chapter 10 material. Next: letting a model read these grids for you (machine learning).

Chapter 11 — Introductory Machine Learning

Mapping burned area from Landsat imagery

We introduce machine-learning **programming** with scikit-learn on a **real remote-sensing task**: teach a model to label each pixel as **burned** or **unburned** from Landsat 8 spectral bands. We train on one date and then test the model on the **same area weeks later** — a powerful lesson in generalisation.

This is an introduction to ML *programming*, **not** spatial statistics (that's your R course). Run in **Python (gem530)**.

Data (`data/`):

- `LC08_L1TP_20190702_image1.tif` — Landsat 8 scene (the **features**, one per band)
- `fires_clip_bi.tif` — binary burned/unburned **labels** for that scene
- `LC08_L1TP_20190819_image2.tif` — same area, ~6 weeks later (**unseen test**)

Learning outcomes

1. Turn a multi-band raster into an ML table of `(pixels, bands)`.
2. Split into **training** and **test** sets and fit a classifier.
3. Evaluate with **accuracy**, a **confusion matrix**, and **kappa**.
4. Predict across a whole image and **map** the result.
5. Test the model on a **new date** and reason about **overfitting/generalisation**.
6. State the **limitations** of the model.

1. Read the imagery and labels (Rasterio)

Each Landsat band becomes a **feature**; the fires raster gives the **label** for every pixel.

```
import numpy as np, rasterio
import matplotlib.pyplot as plt

with rasterio.open("data/LC08_L1TP_20190702_image1.tif") as src:
    landsat = src.read()          # shape: (bands, rows, cols)
    meta = src.meta
with rasterio.open("data/fires_clip_bi.tif") as src:
    fires = src.read(1)          # shape: (rows, cols); 1 = burned, 0 = not

print("landsat:", landsat.shape, "| fires:", fires.shape)
```

```
# quick look: the near-infrared band and the fire labels
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
ax[0].imshow(landsat[4], cmap="Greys"); ax[0].set_title("Landsat NIR band")
ax[1].imshow(fires, cmap="Reds");      ax[1].set_title("Burned (1) / not (0)")
plt.show()
```

🌲 **Why code this?** Supervised learning needs *examples*: imagery (the evidence) plus labels (the truth). Assembling that pair correctly is most of the work in applied remote sensing.

2. Reshape into an ML table

scikit-learn expects **X** as **(samples, features)** — for imagery that means **(pixels, bands)**. We flatten the rows×cols grid and move bands to the last axis.

```
bands, rows, cols = landsat.shape
X = landsat.reshape(bands, rows * cols)    # (bands, pixels)
X = np.swapaxes(X, 0, 1)                   # (pixels, bands) <- what sklearn
wants
y = fires.reshape(rows * cols)              # (pixels,) labels
print("X:", X.shape, "| y:", y.shape, "| burned pixels:", int(y.sum()))
```

🌲 **Why code this?** Models want a table — one row per pixel, one column per band. Reshaping the image into that table is the bridge between a raster and

machine learning.

3. Train / test split & fit a model

We hold out part of the pixels to test on. A **Decision Tree** is a simple, fast first classifier. `random_state` makes it reproducible.

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,
random_state=42)
clf = DecisionTreeClassifier(max_depth=8, random_state=42) # max_depth limits
overfitting
clf.fit(X_train, y_train)
print("model trained on", X_train.shape[0], "pixels")
```

🌲 **Forestry example.** Training on part of a burn and testing on the rest tells you whether the model *learned burn signatures* or just memorised those pixels.

✓ Quick check — why split?

Why fit on a *training* set and score on a *test* set?

▼ ► Show answer

So the score reflects performance on **unseen** data. Scoring on data the model trained on is optimistic (that's *data leakage*).

4. Evaluate on the held-out test pixels

```
from sklearn.metrics import accuracy_score, confusion_matrix, cohen_kappa_score
pred = clf.predict(X_test)
print("accuracy:", round(accuracy_score(y_test, pred), 3))
print("kappa:", round(cohen_kappa_score(y_test, pred), 3))
```

```
print("confusion matrix [rows=actual, cols=pred]:\n", confusion_matrix(y_test,
pred))
```

🌲 **Why code this?** Accuracy alone hides trouble when classes are unbalanced. A confusion matrix shows which class is actually being missed — the burned area, usually.

5. Predict the whole image and map it

Predictions matter most when we can **see them in space** — predict every pixel and reshape back to the image grid (this is 'joining predictions back to geospatial data').

```
pred_image = clf.predict(X).reshape(rows, cols)

fig, ax = plt.subplots(1, 2, figsize=(9, 4))
ax[0].imshow(fires, cmap="Reds");      ax[0].set_title("Actual burned area")
ax[1].imshow(pred_image, cmap="Reds"); ax[1].set_title("Predicted burned area")
plt.show()
```

🌲 **Forestry example.** Predicting the whole scene turns a trained model into a **map** — the deliverable a fire or forest-health program actually uses.

6. The real test: a different date (generalisation)

Our model has only seen **one day**. Let's apply it to the **same area ~6 weeks later**. If accuracy drops a lot, the model **overfit** to that single scene.

```
with rasterio.open("data/LC08_L1TP_20190819_image2.tif") as src:
    later = src.read()
Xlater = np.swapaxes(later.reshape(bands, rows * cols), 0, 1)

acc_later = clf.score(Xlater, y)  # compare to the same fire labels
print("accuracy on the later date:", round(acc_later, 3))
```

```
print("(compare with the test accuracy above – a big drop signals  
overfitting)")
```

🌲 **Why code this?** A model that works on one date may fail on another (different sun angle, phenology, sensor). Testing on a new date is the honest test of transferability.

✅ Quick check — generalisation

Accuracy is 0.89 on the test split but 0.82 on a later date. What does that tell you?

▼ ► Show answer

The model **transfers imperfectly across time** — conditions changed (phenology, sun angle, atmosphere). Report the later-date number too; it's the realistic one.

7. Limitations (always state them)

Be honest about what this model can and cannot do:

- Trained on **one scene, one date, one area** — it may not transfer to other regions/ seasons/sensors.
- Pixels are treated independently (no spatial context).
- Labels themselves have error; class imbalance (few burned pixels) inflates accuracy.
- Predicting over pixels the model trained on overstates performance — trust the held-out and later-date results.

🌲 **Why code this?** Stating limits — sample bias, no spatial validation, one sensor, one season — is what separates a defensible result from an over-claimed one.

🔧 Hands-on activity — Fit and evaluate a small classifier (~20 min)

A small labelled sample of spectral values is built below (forest vs burned). Work individually.

Task 1. Split into train/test (30% test, `random_state=0`). **Task 2.** Fit a `DecisionTreeClassifier` and report the **test** accuracy. **Task 3.** Print the confusion matrix and say which class is harder.

```
import numpy as np, pandas as pd
rng = np.random.default_rng(7)
forest = pd.DataFrame({"red": rng.normal(0.06,0.02,80), "nir":
rng.normal(0.45,0.04,80), "label":"forest"})
burned = pd.DataFrame({"red": rng.normal(0.18,0.03,60), "nir":
rng.normal(0.18,0.04,60), "label":"burned"})
sample = pd.concat([forest, burned], ignore_index=True)
X, y = sample[["red","nir"]], sample["label"]
print(sample["label"].value_counts())

# TODO Task 1: train_test_split(..., test_size=0.3, random_state=0, stratify=y)
# TODO Task 2: fit a DecisionTreeClassifier, print test accuracy
# TODO Task 3: confusion matrix
```

Activity solutions

▼ ► Task 1 — split

```
from sklearn.model_selection import train_test_split
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0,
stratify=y)
print(len(Xtr), "train |", len(Xte), "test")
```

▼ ► Task 2 — fit & score

```
from sklearn.tree import DecisionTreeClassifier
m = DecisionTreeClassifier(random_state=0).fit(Xtr, ytr)
print("test accuracy:", round(m.score(Xte, yte), 3))
```

▼ ► Task 3 — confusion matrix

```

from sklearn.metrics import confusion_matrix
print(m.classes_)
print(confusion_matrix(yte, m.predict(Xte), labels=m.classes_))
# rows = actual, cols = predicted; look for the off-diagonal counts

```

Common errors this chapter (and the fix)

Symptom	Cause	Fix
suspiciously perfect accuracy	scored on training data (leakage)	split before fitting; report the test score
accuracy high but map looks wrong	unbalanced classes	check the confusion matrix, not just accuracy
results change every run	no <code>random_state</code>	set <code>random_state=0</code> for reproducibility
shape error in <code>fit</code>	X and y lengths differ	check <code>X.shape</code> and <code>len(y)</code>

Additional learning (optional)

Bonus 1 — Which band mattered? (feature importance)

Why code this? Knowing *why* a model decided something builds trust — and often confirms the ecology (e.g. NIR drops sharply after fire).

```

import numpy as np, pandas as pd
from sklearn.ensemble import RandomForestClassifier
rng = np.random.default_rng(7)
X = pd.DataFrame({"red": np.r_[rng.normal(0.06,0.02,80),
                               rng.normal(0.18,0.03,60)],
                  "nir": np.r_[rng.normal(0.45,0.04,80),
                               rng.normal(0.18,0.04,60)]})
y = ["forest"]*80 + ["burned"]*60
m = RandomForestClassifier(n_estimators=50, random_state=0).fit(X, y)
for band, imp in zip(X.columns, m.feature_importances_):
    print(f"{band}: {imp:.2f}")

```

🌲 A high NIR importance matches the ecology: healthy canopy reflects NIR strongly, burned ground doesn't.

Bonus 2 — Accuracy can lie: class balance

Why code this? If 95% of pixels are unburned, a model that predicts "unburned" everywhere scores 95% and is useless. Always check the balance.

```
import numpy as np
from sklearn.metrics import accuracy_score, confusion_matrix
truth = np.array(["unburned"]*95 + ["burned"]*5)
lazy = np.array(["unburned"]*100)          # predicts one class for everything
print("accuracy of the lazy model:", accuracy_score(truth, lazy))
print(confusion_matrix(truth, lazy, labels=["burned", "unburned"]))
print("-> it never finds a single burned pixel!")
```

🌲 This is why we report the confusion matrix (and kappa) alongside accuracy.

Packages used this chapter

- **scikit-learn** — the ML workflow: `train_test_split`, a classifier's `.fit()` / `.predict()`, and metrics (accuracy, confusion matrix, kappa).
- **Rasterio + NumPy** — read the imagery and reshape it into an ML table (Chapters 9).
- **Matplotlib** — map the actual vs predicted burned area (Chapter 10). See the [Package Guide](#).

Working with AI (30-second habit)

The classic ML mistake is **data leakage** / testing on training data. Notice we report the **held-out test** and the **later-date** accuracy, not the score on pixels the model trained on. If AI code reports one suspiciously perfect number, check what data it was evaluated on.

Recap & what's next

Raster → ML table · train/test split · fit & predict · accuracy/kappa/confusion · map predictions · **test on a new date** (generalisation) · limitations.

- **Practice:** Chapter 11 Exercise.
- **Next:** the final **project** — your own question, your own data.

Practice exercises

Uses the real Landsat/fires rasters (in the Chapter 11 `data/` folder). Run in **gem530**.

Structure: core · apply · interpret · debug · extension. ~30–60 min, individual.

1. Core

Read `data/LC08_L1TP_20190702_image1.tif` (features) and `data/fires_clip_bi.tif` (labels). Reshape to `X` = (pixels, bands) and `y` = (pixels,). Print both shapes.

```
# TODO – your code here
```

2. Apply

First take a **random sample of ~200,000 pixels** for training — you don't need all ~5 million: `rng = np.random.default_rng(0)` then `samp = rng.choice(len(X), 200_000, replace=False)`. Split that sample 70/30 (`random_state=0`), fit a `DecisionTreeClassifier(max_depth=8)`, and print the **test** accuracy.

```
# TODO – your code here
```

3. Interpret

Apply the model to the **later** image `LC08_L1TP_20190819_image2.tif` and compare its accuracy (vs the fire labels) to the test accuracy. Comment on generalisation.

```
# TODO – your code here
```

4. Debug

This reshape is wrong (bands end up as samples) so the model fails. Fix it.

```
X = img.reshape(b, r*c) # missing the swap to (pixels, bands)
```

```
# TODO – your code here
```

5. Extension

Fit a `RandomForestClassifier(n_estimators=50, random_state=0)` and compare its test accuracy to the Decision Tree.

```
# TODO – your code here
```

Going further — advanced (optional)

Optional extension — for when you've finished the Chapter 11 exercise and want more.

You'll practise

1. Engineer a new **feature** (NDVI) from raw bands.
2. Estimate real performance with **cross-validation**.
3. Read which inputs matter via **feature importances**.
4. Inspect errors with a **confusion matrix**.

1. A synthetic 'Landsat' sample

To stay fast and self-contained we simulate pixels where *burned* areas have low NDVI. (The same steps apply to the real Chapter-11 scenes.)

```
import numpy as np
rng = np.random.default_rng(0)
n = 2000
red = rng.uniform(0.10, 0.50, n)
nir = rng.uniform(0.10, 0.60, n)
ndvi = (nir - red) / (nir + red)          # engineered feature
burned = (ndvi < 0.10).astype(int)
flip = rng.random(n) < 0.05              # 5% label noise, like real data
burned = np.where(flip, 1 - burned, burned)
X = np.column_stack([red, nir, ndvi])
y = burned
print("burned pixels:", int(y.sum()), "of", n)
```

2. Cross-validation

A single train/test split can be lucky. **5-fold CV** trains and tests five times for an honest average.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
```

```
rf = RandomForestClassifier(n_estimators=50, random_state=0)
scores = cross_val_score(rf, X, y, cv=5)
print("fold accuracies:", scores.round(3))
print("mean CV accuracy:", round(scores.mean(), 3))
```

✓ Quick check — cross-validation

Why report the mean of 5-fold cross-validation instead of one train/test split?

▼ ✓ Show answer

One split can be lucky or unlucky. Cross-validation averages over several splits, giving a more **reliable** estimate of how the model generalises.

3. Which features matter?

After fitting, `feature_importances_` ranks the inputs.

```
rf.fit(X, y)
for name, imp in zip(["red", "nir", "ndvi"], rf.feature_importances_):
    print(f"{name:>5}: {imp:.3f}")
```

4. Confusion matrix

Accuracy hides *which* class is wrong. The matrix shows it: rows = actual, columns = predicted.

```
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)
rf.fit(Xtr, ytr)
print(confusion_matrix(yte, rf.predict(Xte)))
print("(rows = actual [not-burned, burned], cols = predicted)")
```

🌲 **Why code this?** Good remote-sensing ML is mostly *features and honesty*: NDVI often beats raw bands, cross-validation stops you fooling yourself, and a

confusion matrix reveals whether you're missing the burned pixels that actually matter for a fire-mapping product.

🌟 Challenge

Fit a `DecisionTreeClassifier(max_depth=4)` on the same `X, y` and compare its 5-fold CV mean accuracy with the random forest above.

▼ 🌟 Show one solution

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(max_depth=4, random_state=0)
dt_scores = cross_val_score(dt, X, y, cv=5)
print("decision tree CV mean:", round(dt_scores.mean(), 3))
```

🎓 Nicely done

You've gone a step beyond the Chapter 11 material. That's the whole course — from first variable to a validated classifier. Well done.